PyTorch 로 배우는 기초 AI v1.0

초보자를 위한 필수 가이드

저자*: jw*

이 책은 PyTorch 로 기본적인 신경망 개념을 다룹니다. 각 장에는 다음이 포함됩니다:

- 완전한 소스 코드 - 독립적인 Python 스크립트
- 실제 출력 - 코드 실행 결과
- 핵심 개념 - 중요한 내용 정리

PyTorch 2.8.0 기반

목차

1. 신경망 분류
2. 신경망 회귀
3. 비트코인 가격 예측
4. 활성화 함수가 중요한 이유
5. 패턴 분류 (간소화된 MNIST)
6. Softmax & Cross Entropy 심층 분석
7. Top-K & Temperature 샘플링
8. 토큰 선택: Greedy vs 샘플링
9. 트랜스포머 학습 및 추론 흐름

제 1 장: 신경망 분류

Neural Network Classification

이진 분류*: (x, y)* 좌표로 점을 분류하기

핵심 개념

- nn.Module 기본 클래스
- 이진 분류를 위한 BCELoss
- 확률 출력을 위한 Sigmoid 활성화
- SGD 옵티마이저
- 추론을 위한 model.eval()

소스 코드

basic_neural_network.py

```
"""
Basic Neural Network Example
============================
A simple classification network that learns to classify points
based on their (x, y) coordinates.

Goal: Learn to classify if a point is in the "upper-right" region
      (where x > 0.5 AND y > 0.5) → class 1, otherwise → class 0
"""
```

```python
import torch
import torch.nn as nn

# ============================================================
# STEP 1: Prepare Training Data (inline, 5 samples)
# ============================================================
# Each row: [x, y] coordinates
# Label: 1 if point is in upper-right (x>0.5, y>0.5), else 0

X_train = torch.tensor(
    [
        [0.1, 0.2],  # lower-left region → 0
        [0.8, 0.9],  # upper-right region → 1
        [0.2, 0.8],  # upper-left region → 0
        [0.9, 0.7],  # upper-right region → 1
        [0.3, 0.3],  # lower-left region → 0
    ],
    dtype=torch.float32,
)

y_train = torch.tensor(
    [
        [0],  # class 0
        [1],  # class 1
        [0],  # class 0
        [1],  # class 1
        [0],  # class 0
    ],
    dtype=torch.float32,
)

print("=" * 50)
print("STEP 1: Training Data")
print("=" * 50)
print(f"Input shape: {X_train.shape}")  # [5, 2]
print(f"Label shape: {y_train.shape}")  # [5, 1]
print(f"\nTraining samples:")
for i, (x, y) in enumerate(zip(X_train, y_train)):
    print(f"  Sample {i+1}: ({x[0]:.1f}, {x[1]:.1f}) → class {int(y[0])}")


# ============================================================
# STEP 2: Define Neural Network
# ============================================================
class SimpleClassifier(nn.Module):
    """
    Network architecture:
        Input(2) → Hidden(4) → ReLU → Output(1) → Sigmoid

    - Input: 2 features (x, y coordinates)
    - Hidden: 4 neurons with ReLU activation
    - Output: 1 value (probability of class 1)
    """
```

```python
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(2, 4)  # 2 inputs → 4 hidden
        self.output = nn.Linear(4, 1)  # 4 hidden → 1 output
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.hidden(x)  # Linear transformation
        x = self.relu(x)  # Non-linear activation
        x = self.output(x)  # Linear transformation
        x = self.sigmoid(x)  # Squash to [0, 1]
        return x


model = SimpleClassifier()

print("\n" + "=" * 50)
print("STEP 2: Network Architecture")
print("=" * 50)
print(model)
total_params = sum(p.numel() for p in model.parameters())
print(f"\nTotal parameters: {total_params}")


# ============================================================
# STEP 3: Training Setup
# ============================================================
criterion = nn.BCELoss()  # Binary Cross-Entropy for classification
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)

print("\n" + "=" * 50)
print("STEP 3: Training")
print("=" * 50)


# ============================================================
# STEP 4: Training Loop
# ============================================================
epochs = 100

for epoch in range(epochs):
    # Forward pass: compute predictions
    predictions = model(X_train)

    # Compute loss
    loss = criterion(predictions, y_train)

    # Backward pass: compute gradients
    optimizer.zero_grad()  # Clear old gradients
    loss.backward()  # Compute new gradients

    # Update weights
```

```python
    optimizer.step()

    # Print progress every 20 epochs
    if (epoch + 1) % 20 == 0:
        accuracy = ((predictions > 0.5) == y_train).float().mean()
        print(
            f"Epoch {epoch+1:3d}: Loss = {loss.item():.4f}, "
            f"Accuracy = {accuracy.item()*100:.1f}%"
        )


# ============================================================
# STEP 5: Inference (Test on New Data)
# ============================================================
print("\n" + "=" * 50)
print("STEP 4: Inference on New Data")
print("=" * 50)

X_test = torch.tensor(
    [
        [0.6, 0.7],  # Should be class 1 (upper-right)
        [0.1, 0.9],  # Should be class 0 (upper-left)
        [0.7, 0.8],  # Should be class 1 (upper-right)
    ],
    dtype=torch.float32,
)

# Switch to evaluation mode (disables dropout, etc.)
model.eval()

# No gradient computation needed for inference
with torch.no_grad():
    predictions = model(X_test)
    predicted_classes = (predictions > 0.5).int()

print("\nTest Results:")
print("-" * 40)
for i, (x, prob, cls) in enumerate(zip(X_test, predictions, predicted_classes)):
    print(
        f"  Point ({x[0]:.1f}, {x[1]:.1f}): "
        f"probability = {prob[0]:.3f} → class {cls[0].item()}"
    )


# ============================================================
# STEP 6: Summary
# ============================================================
print("\n" + "=" * 50)
print("Summary")
print("=" * 50)
print(
    """
Neural Network Classification Pipeline:
```

```
1. Prepare data: inputs (features) and labels (targets)
2. Define model: layers and activation functions
3. Choose loss function and optimizer
4. Train: forward → loss → backward → update weights
5. Inference: model.eval() + torch.no_grad()

Key PyTorch patterns:
- nn.Module: base class for all neural networks
- forward(): defines computation graph
- criterion(pred, target): computes loss
- optimizer.zero_grad(): clears gradients
- loss.backward(): computes gradients
- optimizer.step(): updates weights
"""
)
```

실행 결과

```
==================================================
STEP 1: Training Data
==================================================
Input shape: torch.Size([5, 2])
Label shape: torch.Size([5, 1])

Training samples:
  Sample 1: (0.1, 0.2) → class 0
  Sample 2: (0.8, 0.9) → class 1
  Sample 3: (0.2, 0.8) → class 0
  Sample 4: (0.9, 0.7) → class 1
  Sample 5: (0.3, 0.3) → class 0

==================================================
STEP 2: Network Architecture
==================================================
SimpleClassifier(
  (hidden): Linear(in_features=2, out_features=4, bias=True)
  (output): Linear(in_features=4, out_features=1, bias=True)
  (relu): ReLU()
  (sigmoid): Sigmoid()
)

Total parameters: 17

==================================================
STEP 3: Training
==================================================
Epoch  20: Loss = 0.5443, Accuracy = 60.0%
Epoch  40: Loss = 0.3194, Accuracy = 100.0%
Epoch  60: Loss = 0.1580, Accuracy = 100.0%
Epoch  80: Loss = 0.0964, Accuracy = 100.0%
Epoch 100: Loss = 0.0646, Accuracy = 100.0%

==================================================
```

```
STEP 4: Inference on New Data
==================================================

Test Results:
----------------------------------------
  Point (0.6, 0.7): probability = 0.460 → class 0
  Point (0.1, 0.9): probability = 0.015 → class 0
  Point (0.7, 0.8): probability = 0.708 → class 1

==================================================
Summary
==================================================

Neural Network Classification Pipeline:
1. Prepare data: inputs (features) and labels (targets)
2. Define model: layers and activation functions
3. Choose loss function and optimizer
4. Train: forward → loss → backward → update weights
5. Inference: model.eval() + torch.no_grad()

Key PyTorch patterns:
- nn.Module: base class for all neural networks
- forward(): defines computation graph
- criterion(pred, target): computes loss
- optimizer.zero_grad(): clears gradients
- loss.backward(): computes gradients
- optimizer.step(): updates weights
```

제 2 장: 신경망 회귀

Neural Network Regression

집 크기와 방 개수로 집값 예측하기

핵심 개념

- 회귀를 위한 MSELoss
- 출력층에 활성화 함수 없음
- Adam 옵티마이저
- MAE (평균 절대 오차)
- 연속 값 예측

소스 코드

basic_regression.py

```python
"""
Basic Neural Network Regression Example
========================================
A simple regression network that learns to predict house prices
based on size (sqft) and number of rooms.

Goal: Learn the relationship between house features and price
```

```python
      Price ≈ size × 0.1 + rooms × 5 (simplified model)
"""

import torch
import torch.nn as nn

# ============================================================
# STEP 1: Prepare Training Data (inline, 5 samples)
# ============================================================
# Each row: [size (100 sqft), rooms]
# Label: price (in $10,000 units)

X_train = torch.tensor(
    [
        [10.0, 2.0],  # 1000 sqft, 2 rooms → $20k
        [15.0, 3.0],  # 1500 sqft, 3 rooms → $30k
        [20.0, 4.0],  # 2000 sqft, 4 rooms → $40k
        [12.0, 2.0],  # 1200 sqft, 2 rooms → $22k
        [18.0, 3.0],  # 1800 sqft, 3 rooms → $33k
    ],
    dtype=torch.float32,
)

y_train = torch.tensor(
    [
        [20.0],
        [30.0],
        [40.0],
        [22.0],
        [33.0],
    ],
    dtype=torch.float32,
)

print("=" * 50)
print("STEP 1: Training Data")
print("=" * 50)
print(f"Input shape: {X_train.shape}")  # [5, 2]
print(f"Target shape: {y_train.shape}")  # [5, 1]
print(f"\nTraining samples:")
for i, (x, y) in enumerate(zip(X_train, y_train)):
    print(f"  House {i+1}: {x[0]*100:.0f} sqft, {x[1]:.0f} rooms " f"→ ${y[0]*10:.0f}k")


# ============================================================
# STEP 2: Define Neural Network
# ============================================================
class HousePricePredictor(nn.Module):
    """
    Network architecture:
        Input(2) → Hidden(4) → ReLU → Output(1)

    - Input: 2 features (size, rooms)
```

```python
    - Hidden: 4 neurons with ReLU activation
    - Output: 1 value (predicted price) - no activation!

    Note: No activation on output layer for regression
          (we want unbounded continuous values)
    """

    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(2, 4)  # 2 inputs → 4 hidden
        self.output = nn.Linear(4, 1)  # 4 hidden → 1 output
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.hidden(x)  # Linear transformation
        x = self.relu(x)  # Non-linear activation
        x = self.output(x)  # Linear (no activation!)
        return x


model = HousePricePredictor()

print("\n" + "=" * 50)
print("STEP 2: Network Architecture")
print("=" * 50)
print(model)
total_params = sum(p.numel() for p in model.parameters())
print(f"\nTotal parameters: {total_params}")


# ============================================================
# STEP 3: Training Setup
# ============================================================
criterion = nn.MSELoss()  # Mean Squared Error for regression
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)

print("\n" + "=" * 50)
print("STEP 3: Training")
print("=" * 50)


# ============================================================
# STEP 4: Training Loop
# ============================================================
epochs = 500

for epoch in range(epochs):
    # Forward pass: compute predictions
    predictions = model(X_train)

    # Compute loss (MSE)
    loss = criterion(predictions, y_train)
```

```python
    # Backward pass: compute gradients
    optimizer.zero_grad()  # Clear old gradients
    loss.backward()  # Compute new gradients

    # Update weights
    optimizer.step()

    # Print progress every 100 epochs
    if (epoch + 1) % 100 == 0:
        mae = torch.abs(predictions - y_train).mean()  # Mean Absolute Error
        print(
            f"Epoch {epoch+1:3d}: MSE Loss = {loss.item():.4f}, "
            f"MAE = ${mae.item()*10:.2f}k"
        )


# ============================================================
# STEP 5: Check Training Results
# ============================================================
print("\n" + "=" * 50)
print("STEP 4: Training Results")
print("=" * 50)

model.eval()
with torch.no_grad():
    train_preds = model(X_train)

print("\nPredictions vs Actual (Training Data):")
print("-" * 45)
for i, (x, actual, pred) in enumerate(zip(X_train, y_train, train_preds)):
    error = pred[0] - actual[0]
    print(
        f"  House {i+1}: Actual ${actual[0]*10:.0f}k, "
        f"Predicted ${pred[0]*10:.1f}k (error: {error*10:+.1f}k)"
    )


# ============================================================
# STEP 6: Inference (Test on New Data)
# ============================================================
print("\n" + "=" * 50)
print("STEP 5: Inference on New Houses")
print("=" * 50)

X_test = torch.tensor(
    [
        [14.0, 3.0],  # 1400 sqft, 3 rooms
        [25.0, 5.0],  # 2500 sqft, 5 rooms
        [8.0, 1.0],  # 800 sqft, 1 room
    ],
    dtype=torch.float32,
)
```

```python
with torch.no_grad():
    predictions = model(X_test)

print("\nNew House Price Predictions:")
print("-" * 45)
for i, (x, pred) in enumerate(zip(X_test, predictions)):
    print(
        f"  House: {x[0]*100:.0f} sqft, {x[1]:.0f} rooms "
        f"→ Predicted: ${pred[0]*10:.1f}k"
    )


# ============================================================
# STEP 7: Summary
# ============================================================
print("\n" + "=" * 50)
print("Summary")
print("=" * 50)
print(
    """
Classification vs Regression:

┌──────────────────┬──────────────────┬──────────────────┐
│                  │ Classification   │ Regression       │
├──────────────────┼──────────────────┼──────────────────┤
│ Output           │ Class label      │ Continuous value│
│ Output Layer     │ Sigmoid/Softmax  │ None (Linear)    │
│ Loss Function    │ BCE/CrossEntropy│ MSE/MAE          │
│ Metric           │ Accuracy         │ MAE/RMSE         │
└──────────────────┴──────────────────┴──────────────────┘

Key differences in this example:
- No activation on output layer (unbounded predictions)
- MSELoss instead of BCELoss
- MAE (Mean Absolute Error) as evaluation metric
- Adam optimizer (faster convergence than SGD)
"""
)
```

실행 결과

```
==================================================
STEP 1: Training Data
==================================================
Input shape: torch.Size([5, 2])
Target shape: torch.Size([5, 1])

Training samples:
  House 1: 1000 sqft, 2 rooms → $200k
  House 2: 1500 sqft, 3 rooms → $300k
  House 3: 2000 sqft, 4 rooms → $400k
  House 4: 1200 sqft, 2 rooms → $220k
  House 5: 1800 sqft, 3 rooms → $330k
```

```
==================================================
STEP 2: Network Architecture
==================================================
HousePricePredictor(
  (hidden): Linear(in_features=2, out_features=4, bias=True)
  (output): Linear(in_features=4, out_features=1, bias=True)
  (relu): ReLU()
)

Total parameters: 17

==================================================
STEP 3: Training
==================================================
Epoch 100: MSE Loss = 437.7308, MAE = $195.99k
Epoch 200: MSE Loss = 202.4545, MAE = $122.01k
Epoch 300: MSE Loss = 101.2087, MAE = $77.80k
Epoch 400: MSE Loss = 65.8989, MAE = $71.01k
Epoch 500: MSE Loss = 56.1345, MAE = $67.18k

==================================================
STEP 4: Training Results
==================================================

Predictions vs Actual (Training Data):
----------------------------------------------
  House 1: Actual $200k, Predicted $274.2k (error: +74.2k)
  House 2: Actual $300k, Predicted $274.2k (error: -25.8k)
  House 3: Actual $400k, Predicted $274.2k (error: -125.8k)
  House 4: Actual $220k, Predicted $274.2k (error: +54.2k)
  House 5: Actual $330k, Predicted $274.2k (error: -55.8k)

==================================================
STEP 5: Inference on New Houses
==================================================

New House Price Predictions:
----------------------------------------------
  House: 1400 sqft, 3 rooms → Predicted: $274.2k
  House: 2500 sqft, 5 rooms → Predicted: $274.2k
  House: 800 sqft, 1 rooms → Predicted: $274.2k

==================================================
Summary
==================================================

Classification vs Regression:
┌──────────────────┬──────────────────┬──────────────────┐
│                  │ Classification   │ Regression       │
├──────────────────┼──────────────────┼──────────────────┤
│ Output           │ Class label      │ Continuous value│
│ Output Layer     │ Sigmoid/Softmax  │ None (Linear)    │
│ Loss Function    │ BCE/CrossEntropy│ MSE/MAE          │
```

```
│ Metric           │ Accuracy         │ MAE/RMSE         │
└──────────────────┴──────────────────┴──────────────────┘

Key differences in this example:
- No activation on output layer (unbounded predictions)
- MSELoss instead of BCELoss
- MAE (Mean Absolute Error) as evaluation metric
- Adam optimizer (faster convergence than SGD)
```

제 3 장: 비트코인 가격 예측

Bitcoin Price Prediction

시장 지표로 *BTC* 가격을 예측하는 회귀 예제

핵심 개념

- 특성 공학 (거래량, 감성)
- MSELoss 를 이용한 회귀
- 실제 예측 시나리오
- 새 데이터에 대한 모델 추론

소스 코드

basic_bitcoin.py

```python
"""
Bitcoin Price Prediction - Neural Network Regression
=====================================================
A simple regression network that learns to predict Bitcoin price
based on trading volume and market sentiment.

Goal: Learn the relationship between market indicators and BTC price
"""

import torch
import torch.nn as nn

# ============================================================
# STEP 1: Prepare Training Data (inline, 5 samples)
# ============================================================
# Each row: [volume (billions $), sentiment (-1 to +1)]
# Label: BTC price (in $1000 units)

X_train = torch.tensor(
    [
        [20.0, -0.5],  # Low volume, bearish → $25k
        [35.0, 0.2],  # Medium volume, slightly bullish → $42k
        [50.0, 0.8],  # High volume, very bullish → $65k
        [25.0, -0.2],  # Low-medium volume, slightly bearish → $30k
        [45.0, 0.5],  # High volume, bullish → $55k
    ],
    dtype=torch.float32,
```

```python
)

y_train = torch.tensor(
    [
        [25.0],  # $25,000
        [42.0],  # $42,000
        [65.0],  # $65,000
        [30.0],  # $30,000
        [55.0],  # $55,000
    ],
    dtype=torch.float32,
)

print("=" * 55)
print("STEP 1: Training Data")
print("=" * 55)
print(f"Input shape: {X_train.shape}")  # [5, 2]
print(f"Target shape: {y_train.shape}")  # [5, 1]
print(f"\nTraining samples:")
print("-" * 55)
for i, (x, y) in enumerate(zip(X_train, y_train)):
    sentiment = "Bearish" if x[1] < 0 else "Bullish"
    print(
        f"  Day {i+1}: Volume ${x[0]:.0f}B, Sentiment {x[1]:+.1f} ({sentiment}) "
        f"→ ${y[0]:.0f}k"
    )


# ============================================================
# STEP 2: Define Neural Network
# ============================================================
class BitcoinPredictor(nn.Module):
    """
    Network architecture:
        Input(2) → Hidden(8) → ReLU → Output(1)

    - Input: 2 features (volume, sentiment)
    - Hidden: 8 neurons with ReLU activation
    - Output: 1 value (predicted BTC price)
    """

    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(2, 8)  # 2 inputs → 8 hidden
        self.output = nn.Linear(8, 1)  # 8 hidden → 1 output
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.hidden(x)  # Linear transformation
        x = self.relu(x)  # Non-linear activation
        x = self.output(x)  # Linear (no activation for regression)
        return x
```

```python
model = BitcoinPredictor()

print("\n" + "=" * 55)
print("STEP 2: Network Architecture")
print("=" * 55)
print(model)
total_params = sum(p.numel() for p in model.parameters())
print(f"\nTotal parameters: {total_params}")


# ============================================================
# STEP 3: Training Setup
# ============================================================
criterion = nn.MSELoss()  # Mean Squared Error for regression
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)

print("\n" + "=" * 55)
print("STEP 3: Training")
print("=" * 55)


# ============================================================
# STEP 4: Training Loop
# ============================================================
epochs = 500

for epoch in range(epochs):
    # Forward pass
    predictions = model(X_train)

    # Compute loss
    loss = criterion(predictions, y_train)

    # Backward pass
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Print progress
    if (epoch + 1) % 100 == 0:
        mae = torch.abs(predictions - y_train).mean()
        print(
            f"Epoch {epoch+1:3d}: MSE Loss = {loss.item():.4f}, "
            f"MAE = ${mae.item():.2f}k"
        )


# ============================================================
# STEP 5: Check Training Results
# ============================================================
print("\n" + "=" * 55)
print("STEP 4: Training Results")
print("=" * 55)
```

```python
model.eval()
with torch.no_grad():
    train_preds = model(X_train)

print("\nPredictions vs Actual (Training Data):")
print("-" * 55)
for i, (x, actual, pred) in enumerate(zip(X_train, y_train, train_preds)):
    error = pred[0] - actual[0]
    print(
        f"  Day {i+1}: Actual ${actual[0]:.0f}k, "
        f"Predicted ${pred[0]:.1f}k (error: {error:+.1f}k)"
    )


# ============================================================
# STEP 6: Inference (Predict New Market Conditions)
# ============================================================
print("\n" + "=" * 55)
print("STEP 5: Inference - New Market Scenarios")
print("=" * 55)

X_test = torch.tensor(
    [
        [30.0, 0.0],  # Medium volume, neutral
        [60.0, 0.9],  # Very high volume, extremely bullish
        [15.0, -0.8],  # Very low volume, very bearish
        [40.0, 0.3],  # Medium-high volume, moderately bullish
    ],
    dtype=torch.float32,
)

scenarios = [
    "Medium volume, Neutral sentiment",
    "Very high volume, Extremely bullish",
    "Very low volume, Very bearish",
    "Medium-high volume, Moderately bullish",
]

with torch.no_grad():
    predictions = model(X_test)

print("\nBitcoin Price Predictions:")
print("-" * 55)
for scenario, x, pred in zip(scenarios, X_test, predictions):
    print(f"  {scenario}")
    print(f"    Volume: ${x[0]:.0f}B, Sentiment: {x[1]:+.1f}")
    print(f"    → Predicted BTC: ${pred[0]:.1f}k (${pred[0]*1000:.0f})")
    print()


# ============================================================
# STEP 7: Summary
# ============================================================
```

```python
print("=" * 55)
print("Summary")
print("=" * 55)
print(
    """
Bitcoin Price Prediction Model:

┌─────────────────────────────────────────────────────┐
│ Input Features:                                     │
│   • Trading Volume (in billions $)                  │
│   • Market Sentiment (-1 bearish to +1 bullish)     │
├─────────────────────────────────────────────────────┤
│ Output:                                             │
│   • Predicted BTC price (in $1000 units)            │
├─────────────────────────────────────────────────────┤
│ Key Insights:                                       │
│   • Higher volume + bullish sentiment → higher BTC  │
│   • Lower volume + bearish sentiment → lower BTC    │
│   • Neural network learns non-linear patterns       │
└─────────────────────────────────────────────────────┘

Note: This is a simplified educational example.
Real BTC prediction requires many more features!
"""
)
```

실행 결과

```
=======================================================
STEP 1: Training Data
=======================================================
Input shape: torch.Size([5, 2])
Target shape: torch.Size([5, 1])

Training samples:
-------------------------------------------------------
  Day 1: Volume $20B, Sentiment -0.5 (Bearish) → $25k
  Day 2: Volume $35B, Sentiment +0.2 (Bullish) → $42k
  Day 3: Volume $50B, Sentiment +0.8 (Bullish) → $65k
  Day 4: Volume $25B, Sentiment -0.2 (Bearish) → $30k
  Day 5: Volume $45B, Sentiment +0.5 (Bullish) → $55k

=======================================================
STEP 2: Network Architecture
=======================================================
BitcoinPredictor(
  (hidden): Linear(in_features=2, out_features=8, bias=True)
  (output): Linear(in_features=8, out_features=1, bias=True)
  (relu): ReLU()
)

Total parameters: 33
```

```
============================================================
STEP 3: Training
============================================================
Epoch 100: MSE Loss = 2.0512, MAE = $1.31k
Epoch 200: MSE Loss = 1.9212, MAE = $1.30k
Epoch 300: MSE Loss = 1.9209, MAE = $1.30k
Epoch 400: MSE Loss = 1.9209, MAE = $1.30k
Epoch 500: MSE Loss = 1.9209, MAE = $1.30k

============================================================
STEP 4: Training Results
============================================================

Predictions vs Actual (Training Data):
------------------------------------------------------------
  Day 1: Actual $25k, Predicted $23.7k (error: -1.3k)
  Day 2: Actual $42k, Predicted $43.5k (error: +1.5k)
  Day 3: Actual $65k, Predicted $63.1k (error: -1.9k)
  Day 4: Actual $30k, Predicted $30.5k (error: +0.5k)
  Day 5: Actual $55k, Predicted $56.3k (error: +1.3k)

============================================================
STEP 5: Inference - New Market Scenarios
============================================================

Bitcoin Price Predictions:
------------------------------------------------------------
  Medium volume, Neutral sentiment
    Volume: $30B, Sentiment: +0.0
    → Predicted BTC: $37.0k ($36978)

  Very high volume, Extremely bullish
    Volume: $60B, Sentiment: +0.9
    → Predicted BTC: $75.4k ($75357)

  Very low volume, Very bearish
    Volume: $15B, Sentiment: -0.8
    → Predicted BTC: $16.9k ($16870)

  Medium-high volume, Moderately bullish
    Volume: $40B, Sentiment: +0.3
    → Predicted BTC: $49.8k ($49771)

============================================================
Summary
============================================================

Bitcoin Price Prediction Model:
┌──────────────────────────────────────────────────────────┐
│ Input Features:                                          │
│   • Trading Volume (in billions $)                       │
│   • Market Sentiment (-1 bearish to +1 bullish)          │
├──────────────────────────────────────────────────────────┤
```

```
│ Output:                                                  │
│   • Predicted BTC price (in $1000 units)                 │
├──────────────────────────────────────────────────────────┤
│ Key Insights:                                            │
│   • Higher volume + bullish sentiment → higher BTC       │
│   • Lower volume + bearish sentiment → lower BTC         │
│   • Neural network learns non-linear patterns            │
└──────────────────────────────────────────────────────────┘

Note: This is a simplified educational example.
Real BTC prediction requires many more features!
```

제 4 장: 활성화 함수가 중요한 이유

Why Activation Functions Matter

XOR 문제를 통해 비선형 활성화가 필요한 이유 설명

핵심 개념

- 활성화 없이 선형 층이 축소됨
- ReLU 가 선형성을 깨뜨림
- XOR 문제 (비선형)
- 활성화 있음 vs 없음 비교
- 주요 활성화 함수들

소스 코드

basic_activation.py

```python
"""
Why Activation Functions Are Necessary
======================================
Without activation: Multiple layers = 1 linear layer (useless!)
With activation: Network can learn NON-LINEAR patterns

Example: XOR Problem (cannot be solved by linear model)
  Input (0,0) → 0
  Input (0,1) → 1
  Input (1,0) → 1
  Input (1,1) → 0
"""

import torch
import torch.nn as nn

torch.manual_seed(42)

print("=" * 65)
print("WHY ACTIVATION FUNCTIONS ARE NECESSARY")
print("=" * 65)
```

```python
# ============================================================
# STEP 1: The XOR Problem (Non-Linear!)
# ============================================================
print("\n" + "=" * 65)
print("STEP 1: The XOR Problem")
print("=" * 65)

# XOR truth table - cannot be separated by a single line!
X = torch.tensor(
    [
        [0.0, 0.0],
        [0.0, 1.0],
        [1.0, 0.0],
        [1.0, 1.0],
    ]
)

Y = torch.tensor(
    [
        [0.0],  # 0 XOR 0 = 0
        [1.0],  # 0 XOR 1 = 1
        [1.0],  # 1 XOR 0 = 1
        [0.0],  # 1 XOR 1 = 0
    ]
)

print(
    """
XOR Truth Table:
┌──────────┬──────────┬──────────┐
│ Input A  │ Input B  │ Output   │
├──────────┼──────────┼──────────┤
│    0     │    0     │    0     │
│    0     │    1     │    1     │
│    1     │    0     │    1     │
│    1     │    1     │    0     │
└──────────┴──────────┴──────────┘

Visual (cannot draw a single straight line to separate 0s and 1s):

    B
    │
  1 │  ●(1)     ○(0)
    │
  0 │  ○(0)     ●(1)
    └─────────────────── A
       0        1

● = Output 1,  ○ = Output 0
"""
)
```

```python
# ============================================================
# STEP 2: Model WITHOUT Activation (Linear Only)
# ============================================================
print("=" * 65)
print("STEP 2: Model WITHOUT Activation (Linear Only)")
print("=" * 65)


class LinearOnly(nn.Module):
    """Multiple linear layers WITHOUT activation = still linear!"""

    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(2, 8)
        self.layer2 = nn.Linear(8, 8)
        self.layer3 = nn.Linear(8, 1)
        # NO activation functions!

    def forward(self, x):
        x = self.layer1(x)  # Linear
        x = self.layer2(x)  # Linear
        x = self.layer3(x)  # Linear
        return torch.sigmoid(x)  # Only for output probability


print(
    """
Architecture (NO activation between layers):
  Input(2) → Linear(8) → Linear(8) → Linear(1) → Sigmoid

Math proof why this fails:
  Layer1: y1 = W1·x + b1
  Layer2: y2 = W2·y1 + b2 = W2·(W1·x + b1) + b2
             = W2·W1·x + W2·b1 + b2
             = W_combined·x + b_combined  ← Still LINEAR!

  No matter how many layers, it's equivalent to ONE linear layer!
"""
)

model_linear = LinearOnly()
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model_linear.parameters(), lr=0.1)

print("Training Linear Model (500 epochs)...")
print("-" * 40)

for epoch in range(500):
    pred = model_linear(X)
    loss = criterion(pred, Y)

    optimizer.zero_grad()
    loss.backward()
```

```python
    optimizer.step()

    if (epoch + 1) % 100 == 0:
        accuracy = ((pred > 0.5) == Y).float().mean()
        print(
            f"  Epoch {epoch+1}: Loss={loss.item():.4f}, Accuracy={accuracy.item()*100:.1f}%"
        )

# Final results
print("\nFinal Predictions (Linear Model):")
print("-" * 40)
with torch.no_grad():
    pred = model_linear(X)
    for i in range(4):
        actual = int(Y[i].item())
        predicted = pred[i].item()
        result = "✓" if (predicted > 0.5) == actual else "✗"
        print(
            f"  ({int(X[i,0])}, {int(X[i,1])}) → {predicted:.3f} (expected {actual}) {result}"
        )


# ============================================================
# STEP 3: Model WITH Activation (Non-Linear)
# ============================================================
print("\n" + "=" * 65)
print("STEP 3: Model WITH Activation (ReLU)")
print("=" * 65)


class WithActivation(nn.Module):
    """Linear layers WITH ReLU activation = can learn non-linear!"""

    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(2, 8)
        self.layer2 = nn.Linear(8, 8)
        self.layer3 = nn.Linear(8, 1)
        self.relu = nn.ReLU()  # Non-linear activation!

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)  # ← Non-linear!
        x = self.layer2(x)
        x = self.relu(x)  # ← Non-linear!
        x = self.layer3(x)
        return torch.sigmoid(x)


print(
    """
Architecture (WITH ReLU activation):
  Input(2) → Linear(8) → ReLU → Linear(8) → ReLU → Linear(1) → Sigmoid
```

```
ReLU function:
  ReLU(x) = max(0, x)

  If x < 0: output = 0  (cuts off negative values)
  If x ≥ 0: output = x  (passes through)

This simple non-linearity allows network to learn complex patterns!
"""
)

torch.manual_seed(123)  # Different seed for better convergence
model_relu = WithActivation()
optimizer = torch.optim.Adam(model_relu.parameters(), lr=0.1)

print("Training ReLU Model (1000 epochs)...")
print("-" * 40)

for epoch in range(1000):
    pred = model_relu(X)
    loss = criterion(pred, Y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 200 == 0:
        accuracy = ((pred > 0.5) == Y).float().mean()
        print(
            f"  Epoch {epoch+1}: Loss={loss.item():.4f}, Accuracy={accuracy.item()*100:.1f}%"
        )

# Final results
print("\nFinal Predictions (ReLU Model):")
print("-" * 40)
with torch.no_grad():
    pred = model_relu(X)
    for i in range(4):
        actual = int(Y[i].item())
        predicted = pred[i].item()
        result = "✓" if (predicted > 0.5) == actual else "✗"
        print(
            f"  ({int(X[i,0])}, {int(X[i,1])}) → {predicted:.3f} (expected {actual}) {result}"
        )


# ============================================================
# STEP 4: Visual Comparison
# ============================================================
print("\n" + "=" * 65)
print("STEP 4: Comparison")
print("=" * 65)

with torch.no_grad():
```

```
    linear_pred = model_linear(X)
    relu_pred = model_relu(X)

print(
    """
Results Comparison:
┌──────────┬──────────┬──────────────┬──────────────┐
│   Input  │ Expected │  Linear Model │   ReLU Model  │
├──────────┼──────────┼──────────────┼──────────────┤"""
)

for i in range(4):
    inp = f"({int(X[i,0])}, {int(X[i,1])})"
    exp = int(Y[i].item())
    lin = linear_pred[i].item()
    rel = relu_pred[i].item()
    lin_correct = "✓" if (lin > 0.5) == exp else "✗"
    rel_correct = "✓" if (rel > 0.5) == exp else "✗"
    print(
        f"│   {inp}   │    {exp}     │   {lin:.3f}    {lin_correct}   │   {rel:.3f}    {rel_cor
    )

print("└──────────┴──────────┴──────────────┴──────────────┘")

linear_acc = ((linear_pred > 0.5) == Y).float().mean().item() * 100
relu_acc = ((relu_pred > 0.5) == Y).float().mean().item() * 100

print(f"\nAccuracy:")
print(f"  Linear Model: {linear_acc:.1f}%")
print(f"  ReLU Model:   {relu_acc:.1f}%")


# ============================================================
# STEP 5: Common Activation Functions
# ============================================================
print("\n" + "=" * 65)
print("STEP 5: Common Activation Functions")
print("=" * 65)

print(
    """
┌──────────┬──────────────┬──────────────────┐
│ Activation │   Formula   │   Use Case       │
├──────────┼──────────────┼──────────────────┤
│  ReLU    │  max(0, x)   │ Hidden layers (most common)│
│  Sigmoid │  1/(1+e^(-x))│ Binary output (0~1)        │
│  Tanh    │  (e^x-e^(-x))/...│ Hidden layers (-1~1)   │
│  Softmax │  e^xi / Σe^xj │ Multi-class output        │
│  GELU    │  x·Φ(x)      │ Transformers              │
└──────────┴──────────────┴──────────────────┘

Why ReLU is popular:
```

```
  1. Simple: max(0, x) - fast to compute
  2. No vanishing gradient for positive values
  3. Sparse activation (zeros help efficiency)
"""
)


# ============================================================
# STEP 6: Summary
# ============================================================
print("=" * 65)
print("Summary")
print("=" * 65)
print(
    """
Why Activation Functions Are Necessary:

┌─────────────────────────────────────────────────────────────┐
│ WITHOUT Activation (Layers Flattened):                      │
│                                                             │
│   Layer1 → Layer2 → Layer3  =  Single Layer                 │
│                                                             │
│   W3 × (W2 × (W1 × x)) = (W3 × W2 × W1) × x = W_flat × x    │
│                                                             │
│   • Multiple layers COLLAPSE into 1 linear layer            │
│   • Adding more layers is USELESS                           │
│   • Can only learn straight lines                           │
│   • FAILS on XOR: 25% accuracy                              │
│                                                             │
├─────────────────────────────────────────────────────────────┤
│ WITH Activation (Can Stack Layers):                         │
│                                                             │
│   Layer1 → ReLU → Layer2 → ReLU → Layer3                    │
│                                                             │
│   ReLU breaks linearity! Each layer adds NEW capability:    │
│                                                             │
│   • Layer 1: Learn simple patterns                          │
│   • Layer 2: Combine patterns → complex features            │
│   • Layer 3: Combine features → final decision              │
│                                                             │
│   • NON-LINEAR: can learn curves, not just lines            │
│   • STACKABLE: more layers = more power                     │
│   • SOLVES XOR: 100% accuracy                               │
│                                                             │
└─────────────────────────────────────────────────────────────┘

Visual:
  WITHOUT activation:     WITH activation:
  ┌─────┐                 ┌─────┐
  │  L1 │                 │  L1 │
  └─────┘                 └─────┘
     │    ← same as →        │ ReLU ← breaks linearity!
```

```
    │        one layer        │
  ┌─┴─┐                     ┌─┴─┐
  │ L2 │                     │ L2 │
  └─┬─┘                     └─┬─┘
    │                         │ ReLU
  ┌─┴─┐                     ┌─┴─┐
  │ L3 │                     │ L3 │
  └───┘                     └───┘
  = L_flat                  = Deep Network!
"""
)
```

실행 결과

```
================================================================
WHY ACTIVATION FUNCTIONS ARE NECESSARY
================================================================

================================================================
STEP 1: The XOR Problem
================================================================

XOR Truth Table:
┌──────────┬──────────┬──────────┐
│ Input A  │ Input B  │ Output   │
├──────────┼──────────┼──────────┤
│    0     │    0     │    0     │
│    0     │    1     │    1     │
│    1     │    0     │    1     │
│    1     │    1     │    0     │
└──────────┴──────────┴──────────┘

Visual (cannot draw a single straight line to separate 0s and 1s):

    B
    │
  1 │  ●(1)     ○(0)
    │
  0 │  ○(0)     ●(1)
    └──────────────── A
       0        1

● = Output 1,  ○ = Output 0

================================================================
STEP 2: Model WITHOUT Activation (Linear Only)
================================================================

Architecture (NO activation between layers):
  Input(2) → Linear(8) → Linear(8) → Linear(1) → Sigmoid

Math proof why this fails:
  Layer1: y1 = W1·x + b1
```

```
  Layer2: y2 = W2·y1 + b2 = W2·(W1·x + b1) + b2
             = W2·W1·x + W2·b1 + b2
             = W_combined·x + b_combined  ← Still LINEAR!

  No matter how many layers, it's equivalent to ONE linear layer!

Training Linear Model (500 epochs)...
----------------------------------------
  Epoch 100: Loss=0.6931, Accuracy=50.0%
  Epoch 200: Loss=0.6931, Accuracy=75.0%
  Epoch 300: Loss=0.6931, Accuracy=50.0%
  Epoch 400: Loss=0.6931, Accuracy=50.0%
  Epoch 500: Loss=0.6931, Accuracy=25.0%

Final Predictions (Linear Model):
----------------------------------------
  (0, 0) → 0.500 (expected 0) ✓
  (0, 1) → 0.500 (expected 1) ✗
  (1, 0) → 0.500 (expected 1) ✗
  (1, 1) → 0.500 (expected 0) ✗

======================================================================
STEP 3: Model WITH Activation (ReLU)
======================================================================

Architecture (WITH ReLU activation):
  Input(2) → Linear(8) → ReLU → Linear(8) → ReLU → Linear(1) → Sigmoid

ReLU function:
  ReLU(x) = max(0, x)

  If x < 0: output = 0  (cuts off negative values)
  If x ≥ 0: output = x  (passes through)

This simple non-linearity allows network to learn complex patterns!

Training ReLU Model (1000 epochs)...
----------------------------------------
  Epoch 200: Loss=0.0000, Accuracy=100.0%
  Epoch 400: Loss=0.0000, Accuracy=100.0%
  Epoch 600: Loss=0.0000, Accuracy=100.0%
  Epoch 800: Loss=0.0000, Accuracy=100.0%
  Epoch 1000: Loss=0.0000, Accuracy=100.0%

Final Predictions (ReLU Model):
----------------------------------------
  (0, 0) → 0.000 (expected 0) ✓
  (0, 1) → 1.000 (expected 1) ✓
  (1, 0) → 1.000 (expected 1) ✓
  (1, 1) → 0.000 (expected 0) ✓

======================================================================
STEP 4: Comparison
======================================================================
```

Results Comparison:

Input	Expected	Linear Model	ReLU Model
(0, 0)	0	0.500 ✓	0.000 ✓
(0, 1)	1	0.500 ✗	1.000 ✓
(1, 0)	1	0.500 ✗	1.000 ✓
(1, 1)	0	0.500 ✗	0.000 ✓

```
Accuracy:
  Linear Model: 25.0%
  ReLU Model:   100.0%

================================================================
STEP 5: Common Activation Functions
================================================================
```

Activation	Formula	Use Case
ReLU	max(0, x)	Hidden layers (most common)
Sigmoid	1/(1+e^(-x))	Binary output (0~1)
Tanh	(e^x-e^(-x))/...	Hidden layers (-1~1)
Softmax	e^xi / Σe^xj	Multi-class output
GELU	x·Φ(x)	Transformers

```
Why ReLU is popular:
  1. Simple: max(0, x) - fast to compute
  2. No vanishing gradient for positive values
  3. Sparse activation (zeros help efficiency)

================================================================
Summary
================================================================

Why Activation Functions Are Necessary:

┌──────────────────────────────────────────────────────────────┐
│ WITHOUT Activation (Layers Flattened):                       │
│                                                              │
│   Layer1 → Layer2 → Layer3  =  Single Layer                  │
│                                                              │
│   W3 × (W2 × (W1 × x)) = (W3 × W2 × W1) × x = W_flat × x     │
│                                                              │
│   • Multiple layers COLLAPSE into 1 linear layer             │
│   • Adding more layers is USELESS                            │
│   • Can only learn straight lines                            │
│   • FAILS on XOR: 25% accuracy                               │
│                                                              │
├──────────────────────────────────────────────────────────────┤
```

```
│ WITH Activation (Can Stack Layers):                                  │
│                                                                      │
│   Layer1 → ReLU → Layer2 → ReLU → Layer3                             │
│                                                                      │
│   ReLU breaks linearity! Each layer adds NEW capability:             │
│                                                                      │
│   • Layer 1: Learn simple patterns                                   │
│   • Layer 2: Combine patterns → complex features                     │
│   • Layer 3: Combine features → final decision                       │
│                                                                      │
│   • NON-LINEAR: can learn curves, not just lines                     │
│   • STACKABLE: more layers = more power                              │
│   • SOLVES XOR: 100% accuracy                                        │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘

Visual:
  WITHOUT activation:     WITH activation:
   ┌─────┐                 ┌─────┐
   │ L1  │                 │ L1  │
   └─────┘                 └─────┘
      │     ← same as →        │ ReLU ← breaks linearity!
      │       one layer        │
   ┌─────┐                 ┌─────┐
   │ L2  │                 │ L2  │
   └─────┘                 └─────┘
      │                       │ ReLU
      │                       │
   ┌─────┐                 ┌─────┐
   │ L3  │                 │ L3  │
   └─────┘                 └─────┘
  = L_flat                = Deep Network!
```

제 5 장: 패턴 분류 (간소화된 MNIST)

Pattern Classification (Simplified MNIST)

이미지 대신 텍스트 패턴을 사용한 다중 클래스 분류

핵심 개념

- 다중 클래스를 위한 CrossEntropyLoss
- Softmax 확률
- 예측을 위한 argmax
- 완전한 학습 워크플로우
- 신뢰도 점수

소스 코드

basic_mnist_simplified.py

```
"""
Simplified MNIST: Pattern Classification
```

```python
========================================
Instead of images, we use simple text patterns to understand
the PyTorch training workflow.

Patterns:
  "aaa" → 0    "fff" → 5
  "bbb" → 1    "ggg" → 6
  "ccc" → 2    "hhh" → 7
  "ddd" → 3    "iii" → 8
  "eee" → 4    "jjj" → 9

This teaches: data prep → model → training → inference
"""

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)

print("=" * 65)
print("SIMPLIFIED MNIST: Pattern → Digit Classification")
print("=" * 65)

# ================================================================
# STEP 1: Define Patterns and Labels
# ================================================================
print("\n" + "=" * 65)
print("STEP 1: Define Patterns")
print("=" * 65)

# Each pattern is a string of 3 characters
# We'll convert each character to its ASCII value as features
patterns = [
    "aaa",
    "bbb",
    "ccc",
    "ddd",
    "eee",  # → 0, 1, 2, 3, 4
    "fff",
    "ggg",
    "hhh",
    "iii",
    "jjj",  # → 5, 6, 7, 8, 9
]
labels = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print("\nPattern → Label mapping:")
print("-" * 30)
for pattern, label in zip(patterns, labels):
    print(f"  '{pattern}' → {label}")


# ================================================================
```

```python
# STEP 2: Convert Patterns to Tensors
# ============================================================
print("\n" + "=" * 65)
print("STEP 2: Convert to Tensors")
print("=" * 65)


def pattern_to_tensor(pattern):
    """Convert pattern string to normalized tensor."""
    # Convert each character to number (a=0, b=1, ..., j=9)
    values = [ord(c) - ord("a") for c in pattern]
    # Normalize to 0-1 range
    normalized = [v / 9.0 for v in values]
    return normalized


# Create training data (3 samples per pattern for more data)
X_list = []
Y_list = []

for pattern, label in zip(patterns, labels):
    for _ in range(3):  # 3 samples per pattern
        X_list.append(pattern_to_tensor(pattern))
        Y_list.append(label)

X_train = torch.tensor(X_list, dtype=torch.float32)
Y_train = torch.tensor(Y_list, dtype=torch.long)

print(f"\nTraining data shape:")
print(f"  X_train: {X_train.shape}  (30 samples, 3 features)")
print(f"  Y_train: {Y_train.shape}  (30 labels)")

print(f"\nSample conversions:")
print("-" * 50)
for i in range(0, 30, 3):  # Show one per pattern
    pattern = patterns[i // 3]
    features = X_train[i].tolist()
    label = Y_train[i].item()
    print(f"  '{pattern}' → features {[f'{f:.2f}' for f in features]} → label {label}")


# ============================================================
# STEP 3: Define the Model
# ============================================================
print("\n" + "=" * 65)
print("STEP 3: Define Model")
print("=" * 65)


class PatternClassifier(nn.Module):
    """
    Simple classifier for pattern recognition.
    Using nn.Sequential to show ReLU between layers clearly.
```

```python
    """

    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(3, 16),  # Layer 1: 3 features → 16 hidden
            nn.ReLU(),  # Activation
            nn.Linear(16, 16),  # Layer 2: 16 → 16 hidden
            nn.ReLU(),  # Activation
            nn.Linear(16, 10),  # Layer 3: 16 → 10 classes
        )

    def forward(self, x):
        return self.network(x)


model = PatternClassifier()

print(f"\nModel architecture:")
print(model)

total_params = sum(p.numel() for p in model.parameters())
print(f"\nTotal parameters: {total_params}")

print(
    """
Flow:
  Input [3] → Linear(16) → ReLU → Linear(16) → ReLU → Linear(10)
                     ↑              ↑
              Activation!    Activation!

  Output: 10 logits (one per digit 0-9)
  Prediction: argmax of logits
"""
)


# ============================================================
# STEP 4: Training Setup
# ============================================================
print("=" * 65)
print("STEP 4: Training Setup")
print("=" * 65)

criterion = nn.CrossEntropyLoss()  # For multi-class classification
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

print(
    f"""
Loss function: CrossEntropyLoss
  - Combines LogSoftmax + NLLLoss
  - Input: raw logits (10 values)
  - Target: class index (0-9)
```

```
Optimizer: Adam (lr=0.01)
  - Adaptive learning rate
  - Momentum-based updates
"""
)


# ============================================================
# STEP 5: Training Loop
# ============================================================
print("=" * 65)
print("STEP 5: Training Loop")
print("=" * 65)

epochs = 100

print(f"\nTraining for {epochs} epochs...")
print("-" * 65)

for epoch in range(epochs):
    # Forward pass
    logits = model(X_train)  # (30, 10)

    # Compute loss
    loss = criterion(logits, Y_train)

    # Backward pass
    optimizer.zero_grad()  # Clear gradients
    loss.backward()  # Compute gradients
    optimizer.step()  # Update weights

    # Calculate accuracy
    predictions = logits.argmax(dim=1)  # Get predicted class
    correct = (predictions == Y_train).sum().item()
    accuracy = correct / len(Y_train) * 100

    # Print progress every 20 epochs
    if (epoch + 1) % 20 == 0:
        print(f"\n  Epoch {epoch+1}/{epochs}")
        print(f"  ├── Loss:     {loss.item():.4f}")
        print(f"  ├── Correct:  {correct}/{len(Y_train)}")
        print(f"  └── Accuracy: {accuracy:.1f}%")


# ============================================================
# STEP 6: Evaluate on Training Data
# ============================================================
print("\n" + "=" * 65)
print("STEP 6: Training Results")
print("=" * 65)

model.eval()  # Set to evaluation mode
```

```python
with torch.no_grad():
    logits = model(X_train)
    predictions = logits.argmax(dim=1)

print("\nPredictions for each pattern:")
print("-" * 50)
for i in range(0, 30, 3):  # One per pattern
    pattern = patterns[i // 3]
    pred = predictions[i].item()
    actual = Y_train[i].item()
    probs = F.softmax(logits[i], dim=0)
    confidence = probs[pred].item() * 100
    match = "✓" if pred == actual else "✗"
    print(
        f"  '{pattern}' → predicted: {pred}, actual: {actual} "
        f"(confidence: {confidence:.1f}%) {match}"
    )


# ============================================================
# STEP 7: Inference on New Data
# ============================================================
print("\n" + "=" * 65)
print("STEP 7: Inference")
print("=" * 65)

print(
    """
Inference steps:
  1. model.eval()           - Set to evaluation mode
  2. torch.no_grad()        - Disable gradient computation
  3. model(input)           - Forward pass
  4. logits.argmax()        - Get predicted class
  5. softmax(logits)        - Get probabilities (optional)
"""
)


def predict(model, pattern):
    """Predict digit from pattern."""
    model.eval()
    with torch.no_grad():
        # Convert pattern to tensor
        x = torch.tensor([pattern_to_tensor(pattern)], dtype=torch.float32)

        # Forward pass
        logits = model(x)

        # Get prediction and probability
        probs = F.softmax(logits, dim=1)
        pred_class = logits.argmax(dim=1).item()
        confidence = probs[0, pred_class].item() * 100
```

```
        return pred_class, confidence


# Test with known patterns
print("Testing with known patterns:")
print("-" * 50)
test_patterns = ["aaa", "ccc", "eee", "ggg", "jjj"]
for pattern in test_patterns:
    pred, conf = predict(model, pattern)
    expected = ord(pattern[0]) - ord("a")
    match = "✓" if pred == expected else "✗"
    print(f"  Input: '{pattern}' → Output: {pred} (confidence: {conf:.1f}%) {match}")

# Test with new/unseen patterns
print("\nTesting with variations:")
print("-" * 50)
new_patterns = ["aab", "bbc", "abc"]  # Mixed patterns
for pattern in new_patterns:
    pred, conf = predict(model, pattern)
    print(f"  Input: '{pattern}' → Output: {pred} (confidence: {conf:.1f}%)")


# ================================================================
# STEP 8: Understanding the Output
# ================================================================
print("\n" + "=" * 65)
print("STEP 8: Understanding Model Output")
print("=" * 65)

# Show detailed output for one example
test_input = "bbb"
x = torch.tensor([pattern_to_tensor(test_input)], dtype=torch.float32)

with torch.no_grad():
    logits = model(x)
    probs = F.softmax(logits, dim=1)

print(f"\nInput: '{test_input}'")
print(f"\nStep-by-step:")
print(f"  1. Features: {x[0].tolist()}")
print(f"  2. Logits (raw output):")
for i in range(10):
    bar = "▇" * int(max(0, logits[0, i].item() + 5))
    print(f"     Digit {i}: {logits[0, i].item():+.2f} {bar}")

print(f"\n  3. Softmax (probabilities):")
for i in range(10):
    bar = "▇" * int(probs[0, i].item() * 20)
    print(f"     Digit {i}: {probs[0, i].item():.4f} {bar}")

print(f"\n  4. Prediction: argmax = {logits.argmax(dim=1).item()}")
```

```python
# ============================================================
# STEP 9: Summary
# ============================================================
print("\n" + "=" * 65)
print("Summary")
print("=" * 65)
print(
    """
PyTorch Training Workflow:
┌──────────────────────────────────────────────────────────┐
│ 1. DATA PREPARATION                                      │
│    • Convert raw data to tensors                         │
│    • X: features (float), Y: labels (long/int)           │
│                                                          │
│ 2. MODEL DEFINITION                                      │
│    • nn.Module subclass                                  │
│    • Define layers in __init__                           │
│    • Define forward() method                             │
│                                                          │
│ 3. TRAINING LOOP                                         │
│    for epoch in range(epochs):                           │
│        logits = model(X)           # Forward pass        │
│        loss = criterion(logits, Y) # Compute loss        │
│        optimizer.zero_grad()       # Clear gradients     │
│        loss.backward()             # Backpropagation     │
│        optimizer.step()            # Update weights      │
│                                                          │
│ 4. INFERENCE                                             │
│    model.eval()                    # Evaluation mode     │
│    with torch.no_grad():           # No gradients needed │
│        output = model(input)       # Forward pass        │
│        prediction = output.argmax()# Get class           │
└──────────────────────────────────────────────────────────┘

Key Functions:
  • nn.CrossEntropyLoss() - Multi-class classification loss
  • optimizer.zero_grad() - Clear old gradients
  • loss.backward()       - Compute gradients
  • optimizer.step()      - Update model parameters
  • model.eval()          - Set evaluation mode
  • torch.no_grad()       - Disable gradient tracking
"""
)
```

실행 결과

```
=================================================================
SIMPLIFIED MNIST: Pattern → Digit Classification
=================================================================

=================================================================
```

```
STEP 1: Define Patterns
==================================================================

Pattern → Label mapping:
-------------------------------
  'aaa' → 0
  'bbb' → 1
  'ccc' → 2
  'ddd' → 3
  'eee' → 4
  'fff' → 5
  'ggg' → 6
  'hhh' → 7
  'iii' → 8
  'jjj' → 9

==================================================================
STEP 2: Convert to Tensors
==================================================================

Training data shape:
  X_train: torch.Size([30, 3])  (30 samples, 3 features)
  Y_train: torch.Size([30])  (30 labels)

Sample conversions:
-----------------------------------------------------
  'aaa' → features ['0.00', '0.00', '0.00'] → label 0
  'bbb' → features ['0.11', '0.11', '0.11'] → label 1
  'ccc' → features ['0.22', '0.22', '0.22'] → label 2
  'ddd' → features ['0.33', '0.33', '0.33'] → label 3
  'eee' → features ['0.44', '0.44', '0.44'] → label 4
  'fff' → features ['0.56', '0.56', '0.56'] → label 5
  'ggg' → features ['0.67', '0.67', '0.67'] → label 6
  'hhh' → features ['0.78', '0.78', '0.78'] → label 7
  'iii' → features ['0.89', '0.89', '0.89'] → label 8
  'jjj' → features ['1.00', '1.00', '1.00'] → label 9

==================================================================
STEP 3: Define Model
==================================================================

Model architecture:
PatternClassifier(
  (network): Sequential(
    (0): Linear(in_features=3, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=16, bias=True)
    (3): ReLU()
    (4): Linear(in_features=16, out_features=10, bias=True)
  )
)

Total parameters: 506
```

```
Flow:
  Input [3] → Linear(16) → ReLU → Linear(16) → ReLU → Linear(10)
                     ↑              ↑
             Activation!    Activation!

  Output: 10 logits (one per digit 0-9)
  Prediction: argmax of logits

================================================================
STEP 4: Training Setup
================================================================

Loss function: CrossEntropyLoss
  - Combines LogSoftmax + NLLLoss
  - Input: raw logits (10 values)
  - Target: class index (0-9)

Optimizer: Adam (lr=0.01)
  - Adaptive learning rate
  - Momentum-based updates

================================================================
STEP 5: Training Loop
================================================================

Training for 100 epochs...
----------------------------------------------------------------

  Epoch 20/100
  ├── Loss:     2.0645
  ├── Correct:  6/30
  └── Accuracy: 20.0%

  Epoch 40/100
  ├── Loss:     1.5899
  ├── Correct:  15/30
  └── Accuracy: 50.0%

  Epoch 60/100
  ├── Loss:     1.1584
  ├── Correct:  24/30
  └── Accuracy: 80.0%

  Epoch 80/100
  ├── Loss:     0.8017
  ├── Correct:  27/30
  └── Accuracy: 90.0%

  Epoch 100/100
  ├── Loss:     0.5245
  ├── Correct:  30/30
  └── Accuracy: 100.0%
```

```
================================================================
STEP 6: Training Results
================================================================

Predictions for each pattern:
----------------------------------------------------
  'aaa' → predicted: 0, actual: 0 (confidence: 77.2%) ✓
  'bbb' → predicted: 1, actual: 1 (confidence: 58.2%) ✓
  'ccc' → predicted: 2, actual: 2 (confidence: 69.4%) ✓
  'ddd' → predicted: 3, actual: 3 (confidence: 69.1%) ✓
  'eee' → predicted: 4, actual: 4 (confidence: 61.7%) ✓
  'fff' → predicted: 5, actual: 5 (confidence: 57.1%) ✓
  'ggg' → predicted: 6, actual: 6 (confidence: 54.7%) ✓
  'hhh' → predicted: 7, actual: 7 (confidence: 47.0%) ✓
  'iii' → predicted: 8, actual: 8 (confidence: 44.2%) ✓
  'jjj' → predicted: 9, actual: 9 (confidence: 68.3%) ✓

================================================================
STEP 7: Inference
================================================================

Inference steps:
  1. model.eval()           - Set to evaluation mode
  2. torch.no_grad()        - Disable gradient computation
  3. model(input)           - Forward pass
  4. logits.argmax()        - Get predicted class
  5. softmax(logits)        - Get probabilities (optional)

Testing with known patterns:
----------------------------------------------------
  Input: 'aaa' → Output: 0 (confidence: 77.2%) ✓
  Input: 'ccc' → Output: 2 (confidence: 69.4%) ✓
  Input: 'eee' → Output: 4 (confidence: 61.7%) ✓
  Input: 'ggg' → Output: 6 (confidence: 54.7%) ✓
  Input: 'jjj' → Output: 9 (confidence: 68.3%) ✓

Testing with variations:
----------------------------------------------------
  Input: 'aab' → Output: 0 (confidence: 62.8%)
  Input: 'bbc' → Output: 1 (confidence: 49.8%)
  Input: 'abc' → Output: 1 (confidence: 58.3%)

================================================================
STEP 8: Understanding Model Output
================================================================

Input: 'bbb'

Step-by-step:
  1. Features: [0.1111111119389534, 0.1111111119389534, 0.1111111119389534]
  2. Logits (raw output):
     Digit 0: +10.03 ═══════════════
     Digit 1: +11.13 ═══════════════
     Digit 2: +10.04 ═══════════════
```

```
     Digit 3: +8.19 ▇▇▇▇▇▇▇▇▇▇▇▇
     Digit 4: +3.71 ▇▇▇▇▇▇▇▇
     Digit 5: -0.69 ▇▇▇▇
     Digit 6: -2.69 ▇▇
     Digit 7: -8.04
     Digit 8: -12.85
     Digit 9: -16.65

  3. Softmax (probabilities):
     Digit 0: 0.1925 ▇▇▇
     Digit 1: 0.5819 ▇▇▇▇▇▇▇▇▇▇
     Digit 2: 0.1948 ▇▇▇
     Digit 3: 0.0305
     Digit 4: 0.0003
     Digit 5: 0.0000
     Digit 6: 0.0000
     Digit 7: 0.0000
     Digit 8: 0.0000
     Digit 9: 0.0000

  4. Prediction: argmax = 1

================================================================
Summary
================================================================

PyTorch Training Workflow:

┌──────────────────────────────────────────────────────────────┐
│ 1. DATA PREPARATION                                          │
│    • Convert raw data to tensors                             │
│    • X: features (float), Y: labels (long/int)               │
│                                                              │
│ 2. MODEL DEFINITION                                          │
│    • nn.Module subclass                                      │
│    • Define layers in __init__                               │
│    • Define forward() method                                 │
│                                                              │
│ 3. TRAINING LOOP                                             │
│    for epoch in range(epochs):                               │
│        logits = model(X)           # Forward pass            │
│        loss = criterion(logits, Y) # Compute loss            │
│        optimizer.zero_grad()       # Clear gradients         │
│        loss.backward()             # Backpropagation         │
│        optimizer.step()            # Update weights          │
│                                                              │
│ 4. INFERENCE                                                 │
│    model.eval()                    # Evaluation mode         │
│    with torch.no_grad():           # No gradients needed     │
│        output = model(input)       # Forward pass            │
│        prediction = output.argmax()# Get class               │
└──────────────────────────────────────────────────────────────┘
```

```
Key Functions:
  • nn.CrossEntropyLoss() - Multi-class classification loss
  • optimizer.zero_grad() - Clear old gradients
  • loss.backward()       - Compute gradients
  • optimizer.step()      - Update model parameters
  • model.eval()          - Set evaluation mode
  • torch.no_grad()       - Disable gradient tracking
```

제 6 장: Softmax & Cross Entropy 심층 분석

Softmax & Cross Entropy Deep Dive

Softmax 와 *Cross Entropy* 손실 단계별 시각화

핵심 개념

- Softmax 공식: exp(xi) / sum(exp(xj))
- Cross entropy: -log(p_correct)
- 그래디언트: softmax(logits) - one_hot(target)
- 이 조합이 인기 있는 이유
- 트랜스포머 형태 (batch, seq, vocab)

소스 코드

basic_softmax_crossentropy.py

```python
"""
Softmax + Cross Entropy: Step-by-Step Visualization
==========================================================
Understanding the magic: gradient = softmax(logits) - target

This example uses transformer-like shapes:
- Batch size: 2 (two sentences)
- Sequence length: 3 (three tokens per sentence)
- Vocabulary size: 4 (four possible words)

Key insight: CrossEntropyLoss combines LogSoftmax + NLLLoss
The gradient is beautifully simple: softmax(logits) - one_hot(target)
"""

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)  # For reproducibility

print("=" * 60)
print("SOFTMAX + CROSS ENTROPY: Step-by-Step Visualization")
print("=" * 60)

# ============================================================
# STEP 1: Create Simple Logits (Raw Model Output)
# ============================================================
```

```python
print("\n" + "=" * 60)
print("STEP 1: Raw Logits (before softmax)")
print("=" * 60)

# Shape: (batch=2, sequence=3, vocab=4)
# Simplified logits for easy understanding
logits = torch.tensor(
    [
        # Sentence 1: 3 tokens, each predicting from 4 words
        [
            [2.0, 1.0, 0.1, 0.1],  # Token 1: word 0 has highest score
            [0.1, 3.0, 0.1, 0.1],  # Token 2: word 1 has highest score
            [0.1, 0.1, 2.5, 0.1],
        ],  # Token 3: word 2 has highest score
        # Sentence 2: 3 tokens
        [
            [0.1, 0.1, 0.1, 2.0],  # Token 1: word 3 has highest score
            [1.5, 1.5, 0.1, 0.1],  # Token 2: words 0,1 tie
            [0.1, 2.0, 1.0, 0.1],
        ],  # Token 3: word 1 has highest score
    ],
    requires_grad=True,
)

print(f"\nLogits shape: {logits.shape}")
print("  (batch=2, sequence=3, vocabulary=4)")
print(f"\nLogits values:")
for b in range(2):
    print(f"\n  Sentence {b+1}:")
    for t in range(3):
        print(f"    Token {t+1}: {logits[b, t].tolist()}")


# ============================================================
# STEP 2: Apply Softmax (Convert to Probabilities)
# ============================================================
print("\n" + "=" * 60)
print("STEP 2: Softmax (convert logits to probabilities)")
print("=" * 60)

# Softmax along vocabulary dimension (dim=-1)
probs = F.softmax(logits, dim=-1)

print(f"\nSoftmax formula: exp(logit_i) / sum(exp(logit_j))")
print(f"\nProbabilities (sum to 1.0 for each token):")
for b in range(2):
    print(f"\n  Sentence {b+1}:")
    for t in range(3):
        p = probs[b, t]
        print(
            f"    Token {t+1}: [{p[0]:.3f}, {p[1]:.3f}, {p[2]:.3f}, {p[3]:.3f}] "
            f"sum={p.sum():.3f}"
        )
```

```python
# ============================================================
# STEP 3: Target Labels (Ground Truth)
# ============================================================
print("\n" + "=" * 60)
print("STEP 3: Target Labels (ground truth)")
print("=" * 60)

# Target: which word is correct for each token position
# Shape: (batch=2, sequence=3)
targets = torch.tensor(
    [
        [0, 1, 2],  # Sentence 1: correct words are 0, 1, 2
        [3, 0, 1],  # Sentence 2: correct words are 3, 0, 1
    ]
)

print(f"\nTargets shape: {targets.shape}")
print(f"\nTarget labels (correct word index for each token):")
for b in range(2):
    print(f"  Sentence {b+1}: {targets[b].tolist()}")

# One-hot encoding for visualization
one_hot = F.one_hot(targets, num_classes=4).float()
print(f"\nOne-hot encoded targets:")
for b in range(2):
    print(f"\n  Sentence {b+1}:")
    for t in range(3):
        print(
            f"    Token {t+1}: {one_hot[b, t].tolist()} "
            f"(word {targets[b, t].item()} is correct)"
        )


# ============================================================
# STEP 4: Cross Entropy Loss (Manual Calculation)
# ============================================================
print("\n" + "=" * 60)
print("STEP 4: Cross Entropy Loss (manual calculation)")
print("=" * 60)

print(f"\nCross Entropy = -log(probability of correct class)")
print(f"\nFor each token:")

total_loss = 0
for b in range(2):
    print(f"\n  Sentence {b+1}:")
    for t in range(3):
        correct_word = targets[b, t].item()
        prob_correct = probs[b, t, correct_word].item()
        token_loss = -torch.log(torch.tensor(prob_correct)).item()
        total_loss += token_loss
        print(f"    Token {t+1}: -log({prob_correct:.4f}) = {token_loss:.4f}")
```

```python
manual_avg_loss = total_loss / 6  # 2 batches × 3 tokens
print(f"\n  Manual average loss: {manual_avg_loss:.4f}")


# ============================================================
# STEP 5: PyTorch CrossEntropyLoss
# ============================================================
print("\n" + "=" * 60)
print("STEP 5: PyTorch CrossEntropyLoss")
print("=" * 60)

# Reshape for CrossEntropyLoss: (N, C) and (N,)
# N = batch × sequence, C = vocabulary
logits_flat = logits.view(-1, 4)  # (6, 4)
targets_flat = targets.view(-1)  # (6,)

criterion = nn.CrossEntropyLoss()
loss = criterion(logits_flat, targets_flat)

print(f"\nLogits reshaped: {logits.shape} → {logits_flat.shape}")
print(f"Targets reshaped: {targets.shape} → {targets_flat.shape}")
print(f"\nPyTorch CrossEntropyLoss: {loss.item():.4f}")
print(f"Manual calculation:       {manual_avg_loss:.4f}")
print(f"Match: {abs(loss.item() - manual_avg_loss) < 0.0001}")


# ============================================================
# STEP 6: Backpropagation - The Beautiful Gradient
# ============================================================
print("\n" + "=" * 60)
print("STEP 6: Backpropagation - The Beautiful Gradient")
print("=" * 60)

# Compute gradients
loss.backward()

print(f"\n*** THE KEY INSIGHT ***")
print(f"Gradient of CrossEntropy w.r.t. logits:")
print(f"  ∂L/∂logits = softmax(logits) - one_hot(target)")
print(f"\nThis is why softmax + cross entropy is so popular!")
print(f"The gradient is simply: (predicted prob) - (1 if correct else 0)")

# Verify the gradient formula
expected_grad = (probs - one_hot) / 6  # Divided by N for mean
actual_grad = logits.grad

print(f"\n" + "-" * 60)
print("Verification: Comparing gradients")
print("-" * 60)

for b in range(2):
    print(f"\n  Sentence {b+1}:")
    for t in range(3):
```

```
        exp = expected_grad[b, t]
        act = actual_grad[b, t]
        match = torch.allclose(exp, act, atol=1e-6)
        print(f"    Token {t+1}:")
        print(
            f"      Expected (softmax - target): {[f'{x:.4f}' for x in exp.tolist()]}"
        )
        print(
            f"      Actual gradient:             {[f'{x:.4f}' for x in act.tolist()]}"
        )
        print(f"      Match: {match}")


# ============================================================
# STEP 7: Intuitive Explanation
# ============================================================
print("\n" + "=" * 60)
print("STEP 7: Intuitive Explanation")
print("=" * 60)

print(
    """
Example: Token 1 of Sentence 1
  - Logits:      [2.0, 1.0, 0.1, 0.1]
  - Softmax:     [0.574, 0.211, 0.086, 0.086]  (probabilities)
  - Target:      [1, 0, 0, 0]  (word 0 is correct)
  - Gradient:    [0.574-1, 0.211-0, 0.086-0, 0.086-0]
               = [-0.426, 0.211, 0.086, 0.086]

Interpretation:
  - Gradient for correct class (word 0): NEGATIVE (-0.426)
    → Push logit UP to increase probability

  - Gradient for wrong classes: POSITIVE (0.211, 0.086, 0.086)
    → Push logits DOWN to decrease probability

The gradient magnitude shows HOW MUCH to adjust:
  - More confident wrong prediction → larger gradient
  - Already correct prediction → smaller gradient
"""
)


# ============================================================
# STEP 8: Simple 2-Layer Network Example
# ============================================================
print("\n" + "=" * 60)
print("STEP 8: Simple 2-Layer Network with Softmax + CrossEntropy")
print("=" * 60)


class SimpleTransformerLike(nn.Module):
    """
```

```python
    Simplified transformer-like model:
    Input embedding → Hidden1 → ReLU → Hidden2 → Logits

    Instead of attention, just uses linear layers.
    """

    def __init__(self, vocab_size=4, hidden_dim=8):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, hidden_dim)
        self.hidden1 = nn.Linear(hidden_dim, hidden_dim)
        self.hidden2 = nn.Linear(hidden_dim, vocab_size)
        self.relu = nn.ReLU()

    def forward(self, x):
        # x: (batch, sequence) - input token indices
        x = self.embedding(x)  # (batch, seq, hidden)
        x = self.hidden1(x)  # (batch, seq, hidden)
        x = self.relu(x)  # (batch, seq, hidden)
        x = self.hidden2(x)  # (batch, seq, vocab) - logits
        return x


# Create model and sample input
model = SimpleTransformerLike(vocab_size=4, hidden_dim=8)
input_tokens = torch.tensor(
    [
        [0, 1, 2],  # Sentence 1 input
        [3, 0, 1],  # Sentence 2 input
    ]
)
target_tokens = torch.tensor(
    [
        [1, 2, 3],  # Sentence 1 target (next word prediction)
        [0, 1, 2],  # Sentence 2 target
    ]
)

print(f"\nModel architecture:")
print(model)
print(f"\nInput tokens shape: {input_tokens.shape}")
print(f"Target tokens shape: {target_tokens.shape}")

# Forward pass
output_logits = model(input_tokens)
print(f"\nOutput logits shape: {output_logits.shape}")
print(f"  (batch=2, sequence=3, vocabulary=4)")

# Compute loss
output_flat = output_logits.view(-1, 4)
target_flat = target_tokens.view(-1)
loss = criterion(output_flat, target_flat)

print(f"\nLoss: {loss.item():.4f}")
```

```python
# Show gradients flow
loss.backward()
print(f"\nGradients computed! Gradient shapes:")
print(f"  embedding.weight.grad: {model.embedding.weight.grad.shape}")
print(f"  hidden1.weight.grad:   {model.hidden1.weight.grad.shape}")
print(f"  hidden2.weight.grad:   {model.hidden2.weight.grad.shape}")


# ============================================================
# STEP 9: Summary
# ============================================================
print("\n" + "=" * 60)
print("Summary")
print("=" * 60)
print(
    """
Softmax + Cross Entropy Pipeline:
┌──────────────────────────────────────────────────────────┐
│  Logits  ──→  Softmax  ──→  Cross Entropy  ──→  Loss    │
│  (raw)       (probs)        (-log(p_correct))            │
└──────────────────────────────────────────────────────────┘

The Beautiful Gradient:
┌──────────────────────────────────────────────────────────┐
│   ∂L/∂logits = softmax(logits) - one_hot(target)         │
│                                                          │
│   For correct class:  gradient = prob - 1  (negative)    │
│   For wrong classes:  gradient = prob - 0  (positive)    │
└──────────────────────────────────────────────────────────┘

Why this matters:
1. Simple gradient → efficient backpropagation
2. Gradient magnitude ∝ prediction error
3. Naturally handles multi-class classification
4. Numerically stable (LogSoftmax + NLLLoss combined)
"""
)
```

실행 결과

```
============================================================
SOFTMAX + CROSS ENTROPY: Step-by-Step Visualization
============================================================

============================================================
STEP 1: Raw Logits (before softmax)
============================================================

Logits shape: torch.Size([2, 3, 4])
  (batch=2, sequence=3, vocabulary=4)

Logits values:
```

```
  Sentence 1:
    Token 1: [2.0, 1.0, 0.10000000149011612, 0.10000000149011612]
    Token 2: [0.10000000149011612, 3.0, 0.10000000149011612, 0.10000000149011612]
    Token 3: [0.10000000149011612, 0.10000000149011612, 2.5, 0.10000000149011612]

  Sentence 2:
    Token 1: [0.10000000149011612, 0.10000000149011612, 0.10000000149011612, 2.0]
    Token 2: [1.5, 1.5, 0.10000000149011612, 0.10000000149011612]
    Token 3: [0.10000000149011612, 2.0, 1.0, 0.10000000149011612]

============================================================
STEP 2: Softmax (convert logits to probabilities)
============================================================

Softmax formula: exp(logit_i) / sum(exp(logit_j))

Probabilities (sum to 1.0 for each token):

  Sentence 1:
    Token 1: [0.600, 0.221, 0.090, 0.090] sum=1.000
    Token 2: [0.047, 0.858, 0.047, 0.047] sum=1.000
    Token 3: [0.071, 0.071, 0.786, 0.071] sum=1.000

  Sentence 2:
    Token 1: [0.103, 0.103, 0.103, 0.690] sum=1.000
    Token 2: [0.401, 0.401, 0.099, 0.099] sum=1.000
    Token 3: [0.090, 0.600, 0.221, 0.090] sum=1.000

============================================================
STEP 3: Target Labels (ground truth)
============================================================

Targets shape: torch.Size([2, 3])

Target labels (correct word index for each token):
  Sentence 1: [0, 1, 2]
  Sentence 2: [3, 0, 1]

One-hot encoded targets:

  Sentence 1:
    Token 1: [1.0, 0.0, 0.0, 0.0] (word 0 is correct)
    Token 2: [0.0, 1.0, 0.0, 0.0] (word 1 is correct)
    Token 3: [0.0, 0.0, 1.0, 0.0] (word 2 is correct)

  Sentence 2:
    Token 1: [0.0, 0.0, 0.0, 1.0] (word 3 is correct)
    Token 2: [1.0, 0.0, 0.0, 0.0] (word 0 is correct)
    Token 3: [0.0, 1.0, 0.0, 0.0] (word 1 is correct)

============================================================
STEP 4: Cross Entropy Loss (manual calculation)
============================================================
```

```
Cross Entropy = -log(probability of correct class)

For each token:

  Sentence 1:
    Token 1: -log(0.5999) = 0.5110
    Token 2: -log(0.8583) = 0.1528
    Token 3: -log(0.7861) = 0.2407

  Sentence 2:
    Token 1: -log(0.6903) = 0.3707
    Token 2: -log(0.4011) = 0.9136
    Token 3: -log(0.5999) = 0.5110

  Manual average loss: 0.4500

============================================================
STEP 5: PyTorch CrossEntropyLoss
============================================================

Logits reshaped: torch.Size([2, 3, 4]) → torch.Size([6, 4])
Targets reshaped: torch.Size([2, 3]) → torch.Size([6])

PyTorch CrossEntropyLoss: 0.4500
Manual calculation:       0.4500
Match: True

============================================================
STEP 6: Backpropagation - The Beautiful Gradient
============================================================

*** THE KEY INSIGHT ***
Gradient of CrossEntropy w.r.t. logits:
  ∂L/∂logits = softmax(logits) - one_hot(target)

This is why softmax + cross entropy is so popular!
The gradient is simply: (predicted prob) - (1 if correct else 0)

------------------------------------------------------------
Verification: Comparing gradients
------------------------------------------------------------

  Sentence 1:
    Token 1:
      Expected (softmax - target): ['-0.0667', '0.0368', '0.0150', '0.0150']
      Actual gradient:             ['-0.0667', '0.0368', '0.0150', '0.0150']
      Match: True
    Token 2:
      Expected (softmax - target): ['0.0079', '-0.0236', '0.0079', '0.0079']
      Actual gradient:             ['0.0079', '-0.0236', '0.0079', '0.0079']
      Match: True
    Token 3:
      Expected (softmax - target): ['0.0119', '0.0119', '-0.0357', '0.0119']
```

```
      Actual gradient:             ['0.0119', '0.0119', '-0.0357', '0.0119']
      Match: True

  Sentence 2:
    Token 1:
      Expected (softmax - target): ['0.0172', '0.0172', '0.0172', '-0.0516']
      Actual gradient:             ['0.0172', '0.0172', '0.0172', '-0.0516']
      Match: True
    Token 2:
      Expected (softmax - target): ['-0.0998', '0.0668', '0.0165', '0.0165']
      Actual gradient:             ['-0.0998', '0.0668', '0.0165', '0.0165']
      Match: True
    Token 3:
      Expected (softmax - target): ['0.0150', '-0.0667', '0.0368', '0.0150']
      Actual gradient:             ['0.0150', '-0.0667', '0.0368', '0.0150']
      Match: True

============================================================
STEP 7: Intuitive Explanation
============================================================

Example: Token 1 of Sentence 1
  - Logits:      [2.0, 1.0, 0.1, 0.1]
  - Softmax:     [0.574, 0.211, 0.086, 0.086]  (probabilities)
  - Target:      [1, 0, 0, 0]  (word 0 is correct)
  - Gradient:    [0.574-1, 0.211-0, 0.086-0, 0.086-0]
               = [-0.426, 0.211, 0.086, 0.086]

Interpretation:
  - Gradient for correct class (word 0): NEGATIVE (-0.426)
    → Push logit UP to increase probability

  - Gradient for wrong classes: POSITIVE (0.211, 0.086, 0.086)
    → Push logits DOWN to decrease probability

The gradient magnitude shows HOW MUCH to adjust:
  - More confident wrong prediction → larger gradient
  - Already correct prediction → smaller gradient


============================================================
STEP 8: Simple 2-Layer Network with Softmax + CrossEntropy
============================================================

Model architecture:
SimpleTransformerLike(
  (embedding): Embedding(4, 8)
  (hidden1): Linear(in_features=8, out_features=8, bias=True)
  (hidden2): Linear(in_features=8, out_features=4, bias=True)
  (relu): ReLU()
)

Input tokens shape: torch.Size([2, 3])
Target tokens shape: torch.Size([2, 3])
```

```
Output logits shape: torch.Size([2, 3, 4])
  (batch=2, sequence=3, vocabulary=4)

Loss: 1.2451

Gradients computed! Gradient shapes:
  embedding.weight.grad: torch.Size([4, 8])
  hidden1.weight.grad:   torch.Size([8, 8])
  hidden2.weight.grad:   torch.Size([4, 8])

============================================================
Summary
============================================================

Softmax + Cross Entropy Pipeline:

┌──────────────────────────────────────────────────────────┐
│  Logits  ──→  Softmax  ──→  Cross Entropy  ──→  Loss    │
│  (raw)       (probs)        (-log(p_correct))            │
└──────────────────────────────────────────────────────────┘

The Beautiful Gradient:

┌──────────────────────────────────────────────────────────┐
│   ∂L/∂logits = softmax(logits) - one_hot(target)        │
│                                                          │
│   For correct class:  gradient = prob - 1  (negative)   │
│   For wrong classes:  gradient = prob - 0  (positive)   │
└──────────────────────────────────────────────────────────┘

Why this matters:
1. Simple gradient → efficient backpropagation
2. Gradient magnitude ∝ prediction error
3. Naturally handles multi-class classification
4. Numerically stable (LogSoftmax + NLLLoss combined)
```

제 7 장: Top-K & Temperature 샘플링

Top-K & Temperature Sampling

LLM 이 텍스트 생성 다양성을 제어하는 방법

핵심 개념

- Temperature 스케일링
- Top-K 필터링
- Top-P (Nucleus) 샘플링
- multinomial 샘플링
- 결정론적 vs 확률적 생성

소스 코드

basic_topk_temperature.py

```python
"""
Top-K Sampling & Temperature: Step-by-Step Visualization
============================================================
Understanding how LLMs generate diverse yet coherent text.

Temperature: Controls randomness
  - Low (0.1): Sharp distribution → deterministic, repetitive
  - Mid (1.0): Normal distribution → balanced
  - High (2.0): Flat distribution → creative, random

Top-K: Limits vocabulary choices
  - Only sample from the K highest probability tokens
  - Prevents selecting very unlikely tokens
"""

import torch
import torch.nn.functional as F

torch.manual_seed(42)

print("=" * 65)
print("TOP-K SAMPLING & TEMPERATURE: Step-by-Step Visualization")
print("=" * 65)

# ============================================================
# STEP 1: Create Logits (Raw Model Output)
# ============================================================
print("\n" + "=" * 65)
print("STEP 1: Raw Logits from Language Model")
print("=" * 65)

# Vocabulary: ["the", "cat", "dog", "sat", "ran", "happy", "blue", "xyz"]
vocab = ["the", "cat", "dog", "sat", "ran", "happy", "blue", "xyz"]
vocab_size = len(vocab)

# Simulated logits for next token prediction
# Model thinks "cat" and "dog" are most likely, "xyz" is unlikely
logits = torch.tensor([2.0, 4.0, 3.5, 1.0, 0.5, 0.2, 0.1, -2.0])

print(f"\nVocabulary: {vocab}")
print(f"Vocabulary size: {vocab_size}")
print(f"\nLogits (raw model output):")
for i, (word, logit) in enumerate(zip(vocab, logits)):
    bar = "▇" * int(max(0, logit + 3))
    print(f"  {i}: '{word:6s}' → {logit:+.1f} {bar}")


# ============================================================
# STEP 2: Temperature Scaling
# ============================================================
print("\n" + "=" * 65)
print("STEP 2: Temperature Scaling")
print("=" * 65)
```

```python
print(f"\nFormula: scaled_logits = logits / temperature")
print(f"\nEffect on probability distribution:")

temperatures = [0.1, 0.5, 1.0, 2.0]

print(f"\n{'Token':<8}", end="")
for t in temperatures:
    print(f"{'T='+str(t):<12}", end="")
print()
print("-" * 60)

# Calculate probabilities for each temperature
prob_by_temp = {}
for temp in temperatures:
    scaled = logits / temp
    probs = F.softmax(scaled, dim=0)
    prob_by_temp[temp] = probs

# Print probabilities
for i, word in enumerate(vocab):
    print(f"'{word}'", end="")
    print(" " * (7 - len(word)), end="")
    for temp in temperatures:
        p = prob_by_temp[temp][i].item()
        print(f"{p:.4f}      ", end="")
    print()

print("-" * 60)
print("Entropy:", end="  ")
for temp in temperatures:
    probs = prob_by_temp[temp]
    entropy = -(probs * torch.log(probs + 1e-10)).sum().item()
    print(f"{entropy:.3f}       ", end="")
print()

print(
    f"""
Interpretation:
  T=0.1: Very sharp → 'cat' dominates (99.9%) → deterministic
  T=0.5: Sharper   → 'cat' and 'dog' likely → focused
  T=1.0: Normal    → Original distribution → balanced
  T=2.0: Flatter   → More options viable → creative/random
"""
)


# ============================================================
# STEP 3: Visualize Temperature Effect
# ============================================================
print("=" * 65)
print("STEP 3: Visualize Temperature Effect")
print("=" * 65)
```

```python
print(f"\nProbability bars (█ = 5%):\n")
for temp in temperatures:
    print(f"Temperature = {temp}:")
    probs = prob_by_temp[temp]
    for i, (word, p) in enumerate(zip(vocab, probs)):
        bars = int(p.item() * 20)  # 20 bars = 100%
        pct = p.item() * 100
        if pct >= 0.1:  # Only show if >= 0.1%
            print(f"  '{word:6s}' {pct:5.1f}% {'█' * bars}")
    print()


# ============================================================
# STEP 4: Top-K Filtering
# ============================================================
print("=" * 65)
print("STEP 4: Top-K Filtering")
print("=" * 65)

print(f"\nTop-K keeps only the K highest probability tokens.")
print(f"All other tokens get probability = 0.\n")

k_values = [2, 3, 5]
temp = 1.0  # Using T=1.0 for this example
probs = prob_by_temp[temp]

for k in k_values:
    print(f"Top-K = {k}:")
    print("-" * 40)

    # Get top-k values and indices
    topk_probs, topk_indices = torch.topk(probs, k)

    # Create filtered distribution
    filtered_probs = torch.zeros_like(probs)
    filtered_probs[topk_indices] = topk_probs

    # Renormalize
    filtered_probs = filtered_probs / filtered_probs.sum()

    print(f"  Original probs → Top-{k} filtered → Renormalized")
    for i, word in enumerate(vocab):
        orig = probs[i].item()
        filt = filtered_probs[i].item()
        if orig > 0.001 or filt > 0:
            status = "✓ kept" if filt > 0 else "✗ removed"
            print(f"  '{word:6s}': {orig:.4f} → {filt:.4f} {status}")

    print(f"\n  Sum after filtering: {filtered_probs.sum():.4f}")
    print()


# ============================================================
```

```python
# STEP 5: Top-K + Temperature Combined
# ============================================================
print("=" * 65)
print("STEP 5: Top-K + Temperature Combined")
print("=" * 65)

print(f"\nTypical LLM sampling: Apply temperature FIRST, then Top-K\n")


def sample_with_topk_temp(logits, temperature, top_k, num_samples=10):
    """Sample tokens using temperature and top-k."""
    # Step 1: Apply temperature
    scaled_logits = logits / temperature

    # Step 2: Convert to probabilities
    probs = F.softmax(scaled_logits, dim=0)

    # Step 3: Top-K filtering
    topk_probs, topk_indices = torch.topk(probs, top_k)

    # Step 4: Create filtered distribution
    filtered_probs = torch.zeros_like(probs)
    filtered_probs[topk_indices] = topk_probs
    filtered_probs = filtered_probs / filtered_probs.sum()

    # Step 5: Sample
    samples = torch.multinomial(filtered_probs, num_samples, replacement=True)

    return filtered_probs, samples


# Test different configurations
configs = [
    (0.3, 2, "Low temp + K=2: Very focused"),
    (1.0, 3, "Normal temp + K=3: Balanced"),
    (1.5, 5, "High temp + K=5: Creative"),
]

print(f"Sampling 10 tokens with different configurations:\n")

for temp, k, desc in configs:
    filtered_probs, samples = sample_with_topk_temp(logits, temp, k, num_samples=10)

    print(f"{desc}")
    print(f"  Temperature={temp}, Top-K={k}")
    print(f"  Probabilities:")
    for i, word in enumerate(vocab):
        p = filtered_probs[i].item()
        if p > 0:
            print(f"    '{word}': {p:.3f}")

    sampled_words = [vocab[idx] for idx in samples]
    print(f"  Samples: {sampled_words}")
```

```python
    # Count occurrences
    from collections import Counter

    counts = Counter(sampled_words)
    print(f"  Counts: {dict(counts)}")
    print()


# ============================================================
# STEP 6: Step-by-Step Sampling Process
# ============================================================
print("=" * 65)
print("STEP 6: Step-by-Step Sampling Process")
print("=" * 65)

print(f"\nDetailed walkthrough with T=1.0, K=3:\n")

temperature = 1.0
top_k = 3

print("Step 6.1: Original logits")
for i, (word, logit) in enumerate(zip(vocab, logits)):
    print(f"  '{word:6s}': {logit:+.2f}")

print(f"\nStep 6.2: Apply temperature (÷ {temperature})")
scaled = logits / temperature
for i, (word, s) in enumerate(zip(vocab, scaled)):
    print(f"  '{word:6s}': {s:+.2f}")

print(f"\nStep 6.3: Softmax → probabilities")
probs = F.softmax(scaled, dim=0)
for i, (word, p) in enumerate(zip(vocab, probs)):
    print(f"  '{word:6s}': {p:.4f}")

print(f"\nStep 6.4: Top-{top_k} filtering")
topk_probs, topk_indices = torch.topk(probs, top_k)
print(f"  Top-{top_k} indices: {topk_indices.tolist()}")
print(f"  Top-{top_k} words: {[vocab[i] for i in topk_indices]}")
print(f"  Top-{top_k} probs: {[f'{p:.4f}' for p in topk_probs.tolist()]}")

print(f"\nStep 6.5: Renormalize (sum to 1.0)")
filtered = torch.zeros_like(probs)
filtered[topk_indices] = topk_probs
filtered = filtered / filtered.sum()
for i, (word, p) in enumerate(zip(vocab, filtered)):
    if p > 0:
        print(f"  '{word:6s}': {p:.4f}")

print(f"\nStep 6.6: Sample from distribution")
print(f"\n  How multinomial sampling works:")
print(f"  ────────────────────────────────────────────────────────")
print(f"  Probability ranges (cumulative):")
print(f"    'the': 0.0000 ~ 0.0777  (if random falls here → pick 'the')")
```

```
print(f"    'cat': 0.0777 ~ 0.6518  (if random falls here → pick 'cat')")
print(f"    'dog': 0.6518 ~ 1.0000  (if random falls here → pick 'dog')")
print(f"  ──────────────────────────────────────────────────────────")
print(f"\n  Visual:")
print(f"    0.0      0.08      0.65      1.0")
print(f"    ├─'the'──┼───────'cat'───────┼──'dog'──┤")
print(f"    │  7.8%  │       57.4%       │  34.8%  │")
print(f"  ──────────────────────────────────────────────────────────")

print(f"\n  Sampling process:")
torch.manual_seed(123)  # For reproducible demo
cumsum = [0.0, 0.0777, 0.6518, 1.0]  # the, cat, dog boundaries
words = ["the", "cat", "dog"]

for i in range(5):
    # Generate random number
    rand_val = torch.rand(1).item()

    # Find which region it falls into
    if rand_val < 0.0777:
        picked = "the"
        region = f"0.00-0.08"
    elif rand_val < 0.6518:
        picked = "cat"
        region = f"0.08-0.65"
    else:
        picked = "dog"
        region = f"0.65-1.00"

    print(f"    Sample {i+1}: random={rand_val:.4f} → in range {region} → '{picked}'")


# ============================================================
# STEP 7: Top-P (Nucleus) Sampling Bonus
# ============================================================
print("\n" + "=" * 65)
print("STEP 7: Bonus - Top-P (Nucleus) Sampling")
print("=" * 65)

print(
    f"""
Top-P keeps smallest set of tokens whose cumulative probability ≥ P

Example with P=0.9:
"""
)

probs = F.softmax(logits, dim=0)
sorted_probs, sorted_indices = torch.sort(probs, descending=True)

print("Sorted by probability:")
cumsum = 0
kept = []
```

```python
for i, (prob, idx) in enumerate(zip(sorted_probs, sorted_indices)):
    cumsum += prob.item()
    word = vocab[idx]
    status = "✓" if cumsum <= 0.9 or len(kept) == 0 else "✗"
    if status == "✓":
        kept.append(word)
    print(f"  {i+1}. '{word:6s}': {prob:.4f} (cumsum: {cumsum:.4f}) {status}")

print(f"\nTop-P=0.9 keeps: {kept}")


# ============================================================
# STEP 8: Summary
# ============================================================
print("\n" + "=" * 65)
print("Summary")
print("=" * 65)
print(
    """
Temperature & Top-K Sampling:

┌──────────────────────────────────────────────────────────┐
│  Temperature (T):                                        │
│    T < 1: Sharper distribution → more deterministic      │
│    T = 1: Original distribution                          │
│    T > 1: Flatter distribution → more random/creative    │
│                                                          │
│  Top-K:                                                  │
│    Keeps only K highest probability tokens               │
│    Prevents selecting very unlikely tokens               │
│                                                          │
│  Top-P (Nucleus):                                        │
│    Keeps smallest set with cumulative prob ≥ P           │
│    Adaptive: more tokens when uncertain, fewer when sure │
├──────────────────────────────────────────────────────────┤
│  Common Settings:                                        │
│    Factual/Code:  T=0.1-0.3, K=10-20   (deterministic)   │
│    Balanced:      T=0.7-0.9, K=40-50   (normal)          │
│    Creative:      T=1.0-1.5, K=50-100  (diverse)         │
└──────────────────────────────────────────────────────────┘

Pipeline:  logits → (÷ temperature) → softmax → top-k → sample
"""
)
```

실행 결과

```
=================================================================
TOP-K SAMPLING & TEMPERATURE: Step-by-Step Visualization
=================================================================

=================================================================
STEP 1: Raw Logits from Language Model
```

```
================================================================

Vocabulary: ['the', 'cat', 'dog', 'sat', 'ran', 'happy', 'blue', 'xyz']
Vocabulary size: 8

Logits (raw model output):
  0: 'the   ' → +2.0 ▇▇▇▇▇
  1: 'cat   ' → +4.0 ▇▇▇▇▇▇▇
  2: 'dog   ' → +3.5 ▇▇▇▇▇▇
  3: 'sat   ' → +1.0 ▇▇▇▇
  4: 'ran   ' → +0.5 ▇▇▇
  5: 'happy ' → +0.2 ▇▇▇
  6: 'blue  ' → +0.1 ▇▇▇
  7: 'xyz   ' → -2.0 ▇

================================================================
STEP 2: Temperature Scaling
================================================================

Formula: scaled_logits = logits / temperature

Effect on probability distribution:

Token   T=0.1       T=0.5       T=1.0       T=2.0
-------------------------------------------------------------
'the'    0.0000      0.0132      0.0725      0.1275
'cat'    0.9933      0.7192      0.5356      0.3466
'dog'    0.0067      0.2646      0.3249      0.2699
'sat'    0.0000      0.0018      0.0267      0.0773
'ran'    0.0000      0.0007      0.0162      0.0602
'happy'  0.0000      0.0004      0.0120      0.0518
'blue'   0.0000      0.0003      0.0108      0.0493
'xyz'    0.0000      0.0000      0.0013      0.0173
-------------------------------------------------------------
Entropy:  0.040       0.667       1.164       1.722

Interpretation:
  T=0.1: Very sharp → 'cat' dominates (99.9%) → deterministic
  T=0.5: Sharper   → 'cat' and 'dog' likely → focused
  T=1.0: Normal    → Original distribution → balanced
  T=2.0: Flatter   → More options viable → creative/random

================================================================
STEP 3: Visualize Temperature Effect
================================================================

Probability bars (▇ = 5%):

Temperature = 0.1:
  'cat   '  99.3% ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇
  'dog   '   0.7%

Temperature = 0.5:
  'the   '   1.3%
```

```
  'cat   '  71.9% ═══════════════
  'dog   '  26.5% ═══════
  'sat   '   0.2%

Temperature = 1.0:
  'the   '   7.2% ═
  'cat   '  53.6% ═══════════
  'dog   '  32.5% ═══════
  'sat   '   2.7%
  'ran   '   1.6%
  'happy '   1.2%
  'blue  '   1.1%
  'xyz   '   0.1%

Temperature = 2.0:
  'the   '  12.8% ═══
  'cat   '  34.7% ═══════
  'dog   '  27.0% ═══════
  'sat   '   7.7% ═
  'ran   '   6.0% ═
  'happy '   5.2% ═
  'blue  '   4.9%
  'xyz   '   1.7%

====================================================================
STEP 4: Top-K Filtering
====================================================================

Top-K keeps only the K highest probability tokens.
All other tokens get probability = 0.

Top-K = 2:
----------------------------------------
  Original probs → Top-2 filtered → Renormalized
  'the   ': 0.0725 → 0.0000 ✗ removed
  'cat   ': 0.5356 → 0.6225 ✓ kept
  'dog   ': 0.3249 → 0.3775 ✓ kept
  'sat   ': 0.0267 → 0.0000 ✗ removed
  'ran   ': 0.0162 → 0.0000 ✗ removed
  'happy ': 0.0120 → 0.0000 ✗ removed
  'blue  ': 0.0108 → 0.0000 ✗ removed
  'xyz   ': 0.0013 → 0.0000 ✗ removed

  Sum after filtering: 1.0000

Top-K = 3:
----------------------------------------
  Original probs → Top-3 filtered → Renormalized
  'the   ': 0.0725 → 0.0777 ✓ kept
  'cat   ': 0.5356 → 0.5741 ✓ kept
  'dog   ': 0.3249 → 0.3482 ✓ kept
  'sat   ': 0.0267 → 0.0000 ✗ removed
  'ran   ': 0.0162 → 0.0000 ✗ removed
  'happy ': 0.0120 → 0.0000 ✗ removed
```

```
  'blue  ': 0.0108 → 0.0000 ✗ removed
  'xyz   ': 0.0013 → 0.0000 ✗ removed

  Sum after filtering: 1.0000

Top-K = 5:
----------------------------------------
  Original probs → Top-5 filtered → Renormalized
  'the   ': 0.0725 → 0.0743 ✓ kept
  'cat   ': 0.5356 → 0.5489 ✓ kept
  'dog   ': 0.3249 → 0.3329 ✓ kept
  'sat   ': 0.0267 → 0.0273 ✓ kept
  'ran   ': 0.0162 → 0.0166 ✓ kept
  'happy ': 0.0120 → 0.0000 ✗ removed
  'blue  ': 0.0108 → 0.0000 ✗ removed
  'xyz   ': 0.0013 → 0.0000 ✗ removed

  Sum after filtering: 1.0000

================================================================
STEP 5: Top-K + Temperature Combined
================================================================

Typical LLM sampling: Apply temperature FIRST, then Top-K

Sampling 10 tokens with different configurations:

Low temp + K=2: Very focused
  Temperature=0.3, Top-K=2
  Probabilities:
    'cat': 0.841
    'dog': 0.159
  Samples: ['cat', 'cat', 'cat', 'cat', 'cat', 'cat', 'dog', 'dog', 'cat', 'cat']
  Counts: {'cat': 8, 'dog': 2}

Normal temp + K=3: Balanced
  Temperature=1.0, Top-K=3
  Probabilities:
    'the': 0.078
    'cat': 0.574
    'dog': 0.348
  Samples: ['cat', 'cat', 'dog', 'dog', 'cat', 'cat', 'cat', 'dog', 'cat', 'cat']
  Counts: {'cat': 7, 'dog': 3}

High temp + K=5: Creative
  Temperature=1.5, Top-K=5
  Probabilities:
    'the': 0.119
    'cat': 0.452
    'dog': 0.324
    'sat': 0.061
    'ran': 0.044
  Samples: ['the', 'dog', 'cat', 'cat', 'the', 'the', 'sat', 'cat', 'cat', 'cat']
  Counts: {'the': 3, 'dog': 1, 'cat': 5, 'sat': 1}
```

```
==================================================================
STEP 6: Step-by-Step Sampling Process
==================================================================

Detailed walkthrough with T=1.0, K=3:

Step 6.1: Original logits
  'the   ': +2.00
  'cat   ': +4.00
  'dog   ': +3.50
  'sat   ': +1.00
  'ran   ': +0.50
  'happy ': +0.20
  'blue  ': +0.10
  'xyz   ': -2.00

Step 6.2: Apply temperature (÷ 1.0)
  'the   ': +2.00
  'cat   ': +4.00
  'dog   ': +3.50
  'sat   ': +1.00
  'ran   ': +0.50
  'happy ': +0.20
  'blue  ': +0.10
  'xyz   ': -2.00

Step 6.3: Softmax → probabilities
  'the   ': 0.0725
  'cat   ': 0.5356
  'dog   ': 0.3249
  'sat   ': 0.0267
  'ran   ': 0.0162
  'happy ': 0.0120
  'blue  ': 0.0108
  'xyz   ': 0.0013

Step 6.4: Top-3 filtering
  Top-3 indices: [1, 2, 0]
  Top-3 words: ['cat', 'dog', 'the']
  Top-3 probs: ['0.5356', '0.3249', '0.0725']

Step 6.5: Renormalize (sum to 1.0)
  'the   ': 0.0777
  'cat   ': 0.5741
  'dog   ': 0.3482

Step 6.6: Sample from distribution

  How multinomial sampling works:
  ──────────────────────────────────────────────────────────
  Probability ranges (cumulative):
    'the': 0.0000 ~ 0.0777  (if random falls here → pick 'the')
    'cat': 0.0777 ~ 0.6518  (if random falls here → pick 'cat')
```

```
    'dog': 0.6518 ~ 1.0000  (if random falls here → pick 'dog')
  ──────────────────────────────────────────────────────────

  Visual:
    0.0      0.08      0.65      1.0
    ├─'the'──┼────────'cat'────────┼──'dog'──┤
    │  7.8%  │       57.4%        │  34.8%  │
  ──────────────────────────────────────────────────────────

  Sampling process:
    Sample 1: random=0.2961 → in range 0.08-0.65 → 'cat'
    Sample 2: random=0.5166 → in range 0.08-0.65 → 'cat'
    Sample 3: random=0.2517 → in range 0.08-0.65 → 'cat'
    Sample 4: random=0.6886 → in range 0.65-1.00 → 'dog'
    Sample 5: random=0.0740 → in range 0.00-0.08 → 'the'

================================================================
STEP 7: Bonus - Top-P (Nucleus) Sampling
================================================================

Top-P keeps smallest set of tokens whose cumulative probability ≥ P

Example with P=0.9:

Sorted by probability:
  1. 'cat   ': 0.5356 (cumsum: 0.5356) ✓
  2. 'dog   ': 0.3249 (cumsum: 0.8605) ✓
  3. 'the   ': 0.0725 (cumsum: 0.9330) ✗
  4. 'sat   ': 0.0267 (cumsum: 0.9597) ✗
  5. 'ran   ': 0.0162 (cumsum: 0.9758) ✗
  6. 'happy ': 0.0120 (cumsum: 0.9878) ✗
  7. 'blue  ': 0.0108 (cumsum: 0.9987) ✗
  8. 'xyz   ': 0.0013 (cumsum: 1.0000) ✗

Top-P=0.9 keeps: ['cat', 'dog']

================================================================
Summary
================================================================

Temperature & Top-K Sampling:
┌──────────────────────────────────────────────────────────────┐
│  Temperature (T):                                            │
│    T < 1: Sharper distribution → more deterministic          │
│    T = 1: Original distribution                              │
│    T > 1: Flatter distribution → more random/creative        │
│                                                              │
│  Top-K:                                                      │
│    Keeps only K highest probability tokens                   │
│    Prevents selecting very unlikely tokens                   │
│                                                              │
│  Top-P (Nucleus):                                            │
│    Keeps smallest set with cumulative prob ≥ P               │
```

```
│    Adaptive: more tokens when uncertain, fewer when sure    │
├──────────────────────────────────────────────────────────────┤
│  Common Settings:                                            │
│    Factual/Code:  T=0.1-0.3, K=10-20   (deterministic)       │
│    Balanced:      T=0.7-0.9, K=40-50   (normal)              │
│    Creative:      T=1.0-1.5, K=50-100  (diverse)             │
└──────────────────────────────────────────────────────────────┘

Pipeline:  logits → (÷ temperature) → softmax → top-k → sample
```

제 8 장: 토큰 선택: Greedy vs 샘플링

Token Selection: Greedy vs Sampling

언어 모델의 최종 토큰 선택 방법

핵심 개념

- argmax (greedy) 선택
- multinomial (랜덤) 샘플링
- 확률 가중 선택
- 각 방법을 사용할 때

소스 코드

basic_token_selection.py

```python
"""
Token Selection: How the Final Token is Chosen
==============================================
After top-k filtering, the final token is RANDOMLY sampled
using torch.multinomial() based on probability weights.

Two modes:
1. Greedy (argmax): Always pick highest probability → deterministic
2. Sampling (multinomial): Random weighted choice → diverse
"""

import torch
import torch.nn.functional as F

torch.manual_seed(42)

print("=" * 60)
print("TOKEN SELECTION: Greedy vs Random Sampling")
print("=" * 60)

# ============================================================
# STEP 1: Setup - Logits after Top-K filtering
# ============================================================
vocab = ["the", "cat", "dog", "sat", "ran"]
```

```python
# After top-k=3 filtering, only 3 tokens remain
logits = torch.tensor([0.5, 4.0, 3.0, -999, -999])  # -999 = filtered out
probs = F.softmax(logits, dim=0)

print("\n" + "=" * 60)
print("STEP 1: After Top-K=3 Filtering")
print("=" * 60)
print("\nRemaining tokens and probabilities:")
for word, p in zip(vocab, probs):
    if p > 0.01:
        print(f"  '{word}': {p:.3f} ({p*100:.1f}%)")

# ============================================================
# STEP 2: Greedy Selection (argmax)
# ============================================================
print("\n" + "=" * 60)
print("STEP 2: GREEDY Selection (argmax)")
print("=" * 60)

print(
    """
Method: torch.argmax(probs)
Always picks the HIGHEST probability token.
Result is DETERMINISTIC - same input = same output.
"""
)

for i in range(5):
    selected_idx = torch.argmax(probs).item()
    print(f"  Run {i+1}: '{vocab[selected_idx]}' (always same)")

# ============================================================
# STEP 3: Random Sampling (multinomial)
# ============================================================
print("\n" + "=" * 60)
print("STEP 3: RANDOM Sampling (multinomial)")
print("=" * 60)

print(
    """
Method: torch.multinomial(probs, num_samples=1)
Randomly picks based on probability weights.
Higher probability = more likely to be chosen.
Result is STOCHASTIC - same input = different outputs.
"""
)

print("Probabilities: cat=73.6%, dog=24.5%, the=1.8%\n")

import time

torch.manual_seed(int(time.time()))  # Use random seed
samples = []
```

```python
for i in range(10):
    selected_idx = torch.multinomial(probs, num_samples=1).item()
    samples.append(vocab[selected_idx])
    print(f"  Run {i+1}: '{vocab[selected_idx]}'")

# Count results
from collections import Counter

counts = Counter(samples)
print(f"\nResults from 10 samples: {dict(counts)}")

# ============================================================
# STEP 4: Visualize Multinomial Sampling
# ============================================================
print("\n" + "=" * 60)
print("STEP 4: How multinomial() Works")
print("=" * 60)

print(
    """
Think of it as a weighted lottery wheel:

    ┌──────────────────────────────────────────┐
    │                                          │
    │     'cat' 73.6%        ██████████        │
    │                        ██████████        │
    │                        ██████████        │
    │     'dog' 24.5%        ████              │
    │     'the'  1.8%        ▌                 │
    │                                          │
    └──────────────────────────────────────────┘

1. Generate random number between 0 and 1
2. See which token's "region" it falls into

Example:
  random = 0.25 → falls in 'cat' region (0.00 - 0.74) → select 'cat'
  random = 0.80 → falls in 'dog' region (0.74 - 0.98) → select 'dog'
  random = 0.99 → falls in 'the' region (0.98 - 1.00) → select 'the'
"""
)

# Manual demonstration
print("Manual simulation:")
print("-" * 40)
cumulative = 0
regions = []
for word, p in zip(vocab, probs):
    if p > 0.01:
        start = cumulative
        cumulative += p.item()
        end = cumulative
        regions.append((word, start, end))
```

```
        print(f"  '{word}': {start:.3f} to {end:.3f}")

print("\n5 random selections:")
for i in range(5):
    r = torch.rand(1).item()
    for word, start, end in regions:
        if start <= r < end:
            print(f"  random={r:.3f} → '{word}'")
            break

# ============================================================
# STEP 5: Temperature + Top-K + Sampling Pipeline
# ============================================================
print("\n" + "=" * 60)
print("STEP 5: Complete LLM Sampling Pipeline")
print("=" * 60)

print(
    """
Full pipeline in most LLMs:

  logits                    [2.0, 4.0, 3.5, 1.0, 0.5]
     ↓
  ÷ temperature             [2.0, 4.0, 3.5, 1.0, 0.5] / 0.7
     ↓
  softmax                   [0.05, 0.55, 0.35, 0.03, 0.02]
     ↓
  top-k filter (k=3)        [0.00, 0.58, 0.37, 0.00, 0.00] + renorm
     ↓
  multinomial sampling      → randomly pick 'cat' or 'dog' or 'the'
     ↓
  FINAL TOKEN               'cat' (this time)
"""
)

# ============================================================
# STEP 6: Greedy vs Sampling Comparison
# ============================================================
print("=" * 60)
print("STEP 6: When to Use Each Method")
print("=" * 60)

print(
    """
┌──────────────────┬──────────────────┬──────────────────────┐
│                  │ Greedy (argmax)  │ Sampling             │
│                  │                  │ (multinomial)        │
├──────────────────┼──────────────────┼──────────────────────┤
│ Selection        │ Highest prob     │ Random by weight     │
│ Output           │ Deterministic    │ Stochastic           │
│ Diversity        │ None             │ High                 │
│ Repetition       │ Common           │ Rare                 │
├──────────────────┼──────────────────┼──────────────────────┤
```

```
│ Use for           │ Factual Q&A      │ Creative writing    │
│                   │ Code generation  │ Brainstorming       │
│                   │ Translation      │ Story generation    │
└───────────────────┴──────────────────┴─────────────────────┘


Code:
  # Greedy
  token = torch.argmax(probs)

  # Sampling
  token = torch.multinomial(probs, num_samples=1)
"""
)
```

실행 결과

```
============================================================
TOKEN SELECTION: Greedy vs Random Sampling
============================================================

============================================================
STEP 1: After Top-K=3 Filtering
============================================================

Remaining tokens and probabilities:
  'the': 0.022 (2.2%)
  'cat': 0.715 (71.5%)
  'dog': 0.263 (26.3%)

============================================================
STEP 2: GREEDY Selection (argmax)
============================================================

Method: torch.argmax(probs)
Always picks the HIGHEST probability token.
Result is DETERMINISTIC - same input = same output.

  Run 1: 'cat' (always same)
  Run 2: 'cat' (always same)
  Run 3: 'cat' (always same)
  Run 4: 'cat' (always same)
  Run 5: 'cat' (always same)

============================================================
STEP 3: RANDOM Sampling (multinomial)
============================================================

Method: torch.multinomial(probs, num_samples=1)
Randomly picks based on probability weights.
Higher probability = more likely to be chosen.
Result is STOCHASTIC - same input = different outputs.

Probabilities: cat=73.6%, dog=24.5%, the=1.8%
```

```
  Run 1: 'dog'
  Run 2: 'cat'
  Run 3: 'cat'
  Run 4: 'dog'
  Run 5: 'cat'
  Run 6: 'dog'
  Run 7: 'cat'
  Run 8: 'dog'
  Run 9: 'cat'
  Run 10: 'dog'

Results from 10 samples: {'dog': 5, 'cat': 5}

============================================================
STEP 4: How multinomial() Works
============================================================

Think of it as a weighted lottery wheel:

     ┌──────────────────────────────────────────┐
     │                                          │
     │     'cat' 73.6%        ██████████        │
     │                        ██████████        │
     │                        ██████████        │
     │     'dog' 24.5%        ████              │
     │     'the'  1.8%        ▌                 │
     │                                          │
     └──────────────────────────────────────────┘

1. Generate random number between 0 and 1
2. See which token's "region" it falls into

Example:
  random = 0.25 → falls in 'cat' region (0.00 - 0.74) → select 'cat'
  random = 0.80 → falls in 'dog' region (0.74 - 0.98) → select 'dog'
  random = 0.99 → falls in 'the' region (0.98 - 1.00) → select 'the'

Manual simulation:
----------------------------------------
  'the': 0.000 to 0.022
  'cat': 0.022 to 0.737
  'dog': 0.737 to 1.000

5 random selections:
  random=0.745 → 'dog'
  random=0.484 → 'cat'
  random=0.107 → 'cat'
  random=0.836 → 'dog'
  random=0.758 → 'dog'

============================================================
STEP 5: Complete LLM Sampling Pipeline
```

```
============================================================

Full pipeline in most LLMs:

  logits                    [2.0, 4.0, 3.5, 1.0, 0.5]
     ↓
  ÷ temperature             [2.0, 4.0, 3.5, 1.0, 0.5] / 0.7
     ↓
  softmax                   [0.05, 0.55, 0.35, 0.03, 0.02]
     ↓
  top-k filter (k=3)        [0.00, 0.58, 0.37, 0.00, 0.00] + renorm
     ↓
  multinomial sampling      → randomly pick 'cat' or 'dog' or 'the'
     ↓
  FINAL TOKEN               'cat' (this time)

============================================================
STEP 6: When to Use Each Method
============================================================
```

	Greedy (argmax)	Sampling (multinomial)
Selection	Highest prob	Random by weight
Output	Deterministic	Stochastic
Diversity	None	High
Repetition	Common	Rare
Use for	Factual Q&A Code generation Translation	Creative writing Brainstorming Story generation

```
Code:
  # Greedy
  token = torch.argmax(probs)

  # Sampling
  token = torch.multinomial(probs, num_samples=1)
```

제 9 장: 트랜스포머 학습 및 추론 흐름

Transformer Training & Inference Flow

트랜스포머와 유사한 모델의 전체 학습 및 생성 흐름

핵심 개념

- 다음 토큰 예측
- 자기회귀 생성
- 배치 처리

- 데이터 흐름 시각화
- 임베딩 -> 은닉층 -> 로짓 파이프라인

소스 코드

basic_transformer_flow.py

```
"""
Basic Transformer-like Model: Understanding the Flow
=====================================================
This example shows the OVERALL FLOW of transformer training/inference,
NOT the internal transformer architecture (attention, etc.)

Task: Next token prediction
  Input:  <sos> hello how are you
  Output: hello how are you <eos>

Using simplified 2-layer neural network to focus on data flow.
"""

import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)

print("=" * 65)
print("BASIC TRANSFORMER FLOW: Training & Inference")
print("=" * 65)

# ============================================================
# STEP 1: Define Vocabulary
# ============================================================
print("\n" + "=" * 65)
print("STEP 1: Vocabulary")
print("=" * 65)

# Simple vocabulary (like GPT with different topics)
vocab = [
    "<pad>",
    "<sos>",
    "<eos>",  # 0, 1, 2: Special tokens
    "the",
    "cat",
    "sat",
    "on",
    "mat",  # 3-7: Sentence 1
    "i",
    "love",
    "pizza",  # 8-10: Sentence 2
    "sky",
    "is",
    "blue",  # 11-13: Sentence 3
]
```

```python
word_to_idx = {word: idx for idx, word in enumerate(vocab)}
idx_to_word = {idx: word for idx, word in enumerate(vocab)}
vocab_size = len(vocab)

print(f"\nVocabulary ({vocab_size} words):")
for idx, word in enumerate(vocab):
    print(f"  {idx}: '{word}'")

# ============================================================
# STEP 2: Prepare Training Data (3 batches)
# ============================================================
print("\n" + "=" * 65)
print("STEP 2: Training Data (batch=3)")
print("=" * 65)

# Three DIFFERENT sentences (like real GPT training)
# Each has UNIQUE starting context so model can learn different patterns
sentences = [
    ["the", "cat", "sat", "on"],  # Sentence 1: predict next tokens
    ["i", "love", "pizza", "<pad>"],  # Sentence 2: shorter, padded
    ["sky", "is", "blue", "<pad>"],  # Sentence 3: shorter, padded
]

# Target: shifted by 1 (next token prediction)
targets = [
    ["cat", "sat", "on", "mat"],  # Target 1: predict cat→sat→on→mat
    ["love", "pizza", "<eos>", "<pad>"],  # Target 2: predict love→pizza→<eos>
    ["is", "blue", "<eos>", "<pad>"],  # Target 3: predict is→blue→<eos>
]

print("\nInput → Target (next token prediction):")
for i, (inp, tgt) in enumerate(zip(sentences, targets)):
    print(f"\n  Batch {i+1}:")
    print(f"    Input:  {inp}")
    print(f"    Target: {tgt}")


# Convert to indices
def to_indices(sentence_list):
    return [[word_to_idx[w] for w in sent] for sent in sentence_list]


X_indices = to_indices(sentences)
Y_indices = to_indices(targets)

# Convert to tensors
X_train = torch.tensor(X_indices)  # (batch=3, seq=5)
Y_train = torch.tensor(Y_indices)  # (batch=3, seq=5)

print(f"\n\nTensor shapes:")
print(f"  X_train: {X_train.shape}  (batch=3, sequence=5)")
print(f"  Y_train: {Y_train.shape}  (batch=3, sequence=5)")
```

```python
print(f"\nX_train (token indices):")
print(X_train)

print(f"\nY_train (target indices):")
print(Y_train)

# ============================================================
# STEP 3: Define Simplified Transformer Model
# ============================================================
print("\n" + "=" * 65)
print("STEP 3: Simplified Model (2 hidden layers)")
print("=" * 65)


class SimplifiedTransformer(nn.Module):
    """
    Simplified model to understand the flow.
    Real transformer uses attention - this uses simple linear layers.

    Flow:
      token_idx → embedding → hidden1 → hidden2 → logits → softmax
    """

    def __init__(self, vocab_size, embed_dim=16, hidden_dim=32):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.hidden1 = nn.Linear(embed_dim, hidden_dim)
        self.hidden2 = nn.Linear(hidden_dim, hidden_dim)
        self.output = nn.Linear(hidden_dim, vocab_size)
        self.relu = nn.ReLU()

    def forward(self, x):
        # x: (batch, sequence) - token indices

        # Step 1: Embedding lookup
        x = self.embedding(x)  # (batch, seq, embed_dim)

        # Step 2: Hidden layer 1
        x = self.hidden1(x)  # (batch, seq, hidden_dim)
        x = self.relu(x)

        # Step 3: Hidden layer 2
        x = self.hidden2(x)  # (batch, seq, hidden_dim)
        x = self.relu(x)

        # Step 4: Output logits
        logits = self.output(x)  # (batch, seq, vocab_size)

        return logits


model = SimplifiedTransformer(vocab_size)
```

```python
print(f"\nModel architecture:")
print(model)

total_params = sum(p.numel() for p in model.parameters())
print(f"\nTotal parameters: {total_params}")

# ============================================================
# STEP 4: Training Setup
# ============================================================
print("\n" + "=" * 65)
print("STEP 4: Training")
print("=" * 65)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

print(f"\nLoss function: CrossEntropyLoss")
print(f"Optimizer: Adam (lr=0.01)")

# ============================================================
# STEP 5: Training Loop
# ============================================================
epochs = 300

print(f"\nTraining for {epochs} epochs...")
print("-" * 65)

for epoch in range(epochs):
    # Forward pass
    logits = model(X_train)  # (batch=3, seq=5, vocab=14)

    # Reshape for CrossEntropyLoss
    # CrossEntropyLoss expects (N, C) and (N,)
    logits_flat = logits.view(-1, vocab_size)  # (15, 14)
    targets_flat = Y_train.view(-1)  # (15,)

    # Compute loss
    loss = criterion(logits_flat, targets_flat)

    # Backward pass
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Print progress every 50 epochs
    if (epoch + 1) % 50 == 0:
        # Calculate accuracy
        predictions = logits.argmax(dim=-1)  # (batch, seq)
        correct = (predictions == Y_train).float().mean()

        print(f"\n  Epoch {epoch+1}/{epochs}")
        print(f"  ├── Loss:     {loss.item():.4f}")
        print(f"  ├── Accuracy: {correct.item()*100:.1f}%")
```

```
        # Show predictions for each batch
        for b in range(3):
            inp_words = [idx_to_word[i.item()] for i in X_train[b]]
            pred_words = [idx_to_word[i.item()] for i in predictions[b]]
            tgt_words = [idx_to_word[i.item()] for i in Y_train[b]]
            match = "✓" if pred_words == tgt_words else "✗"
            # Clean up for display
            inp_str = " ".join([w for w in inp_words if w != "<pad>"])
            pred_str = " ".join([w for w in pred_words if w != "<pad>"])
            print(f"  │   Batch {b+1}: Input: {inp_str}")
            print(f"  │            Output: {pred_str} {match}")

# ============================================================
# STEP 6: Check Training Results
# ============================================================
print("\n" + "=" * 65)
print("STEP 5: Training Results")
print("=" * 65)

model.eval()
with torch.no_grad():
    logits = model(X_train)
    predictions = logits.argmax(dim=-1)

print("\nPredictions vs Targets:")
print("-" * 65)

for b in range(3):
    print(f"\n  Batch {b+1}:")
    inp_words = [idx_to_word[i.item()] for i in X_train[b]]
    tgt_words = [idx_to_word[i.item()] for i in Y_train[b]]
    pred_words = [idx_to_word[i.item()] for i in predictions[b]]

    print(f"    Input:      {inp_words}")
    print(f"    Target:     {tgt_words}")
    print(f"    Predicted:  {pred_words}")

    match = "✓" if pred_words == tgt_words else "✗"
    print(f"    Match: {match}")

# ============================================================
# STEP 7: Inference (Autoregressive Generation)
# ============================================================
print("\n" + "=" * 65)
print("STEP 6: Inference (Autoregressive Generation)")
print("=" * 65)

print(
    """
Autoregressive generation:
  1. Start with <sos>
  2. Predict next token
  3. Append predicted token to input
```

```
  4. Repeat until <eos> or max length
"""
)


def generate(model, start_tokens, max_len=10, temperature=1.0):
    """Generate tokens autoregressively."""
    model.eval()

    # Convert start tokens to indices
    current = [word_to_idx[t] for t in start_tokens]
    generated = list(start_tokens)

    print(f"  Starting: {generated}")

    with torch.no_grad():
        for step in range(max_len):
            # Prepare input
            x = torch.tensor([current])  # (1, seq_len)

            # Forward pass
            logits = model(x)  # (1, seq_len, vocab)

            # Get last token's logits
            last_logits = logits[0, -1, :]  # (vocab,)

            # Apply temperature
            scaled_logits = last_logits / temperature

            # Convert to probabilities
            probs = F.softmax(scaled_logits, dim=0)

            # Sample next token
            next_idx = torch.multinomial(probs, 1).item()
            next_word = idx_to_word[next_idx]

            # Show step
            top3_probs, top3_idx = torch.topk(probs, 3)
            top3_words = [idx_to_word[i.item()] for i in top3_idx]
            print(
                f"  Step {step+1}: top3={list(zip(top3_words, [f'{p:.2f}' for p in top3_probs.to
            )

            # Append to sequence
            current.append(next_idx)
            generated.append(next_word)

            # Stop if <eos>
            if next_word == "<eos>":
                break

    return generated
```

```
print("--- Generation 1 ---")
result1 = generate(model, ["the"], max_len=5, temperature=0.5)
output1 = " ".join([w for w in result1 if w not in ["<eos>", "<pad>"]])
print(f"\n  Input:  the")
print(f"  Output: {output1}")

print("\n--- Generation 2 ---")
result2 = generate(model, ["i"], max_len=5, temperature=0.5)
output2 = " ".join([w for w in result2 if w not in ["<eos>", "<pad>"]])
print(f"\n  Input:  i")
print(f"  Output: {output2}")

print("\n--- Generation 3 ---")
result3 = generate(model, ["sky"], max_len=5, temperature=0.5)
output3 = " ".join([w for w in result3 if w not in ["<eos>", "<pad>"]])
print(f"\n  Input:  sky")
print(f"  Output: {output3}")

# Simple summary
print("\n" + "=" * 65)
print("Inference Summary:")
print("=" * 65)
print(f"  Input: the   →   Output: {output1}")
print(f"  Input: i     →   Output: {output2}")
print(f"  Input: sky   →   Output: {output3}")

# ================================================================
# STEP 8: Visualize Data Flow
# ================================================================
print("\n" + "=" * 65)
print("STEP 7: Data Flow Visualization")
print("=" * 65)

print(
    """
Training Data Flow (3 different sentences in one batch):
─────────────────────────────────────────────────────────────────
Batch 1: "the cat sat on mat"
  Input:   the    cat    sat    on
  Target:  cat    sat    on    mat

Batch 2: "i love pizza"
  Input:    i    love   pizza  <pad>
  Target:  love  pizza  <eos>  <pad>

Batch 3: "sky is blue"
  Input:   sky    is    blue   <pad>
  Target:   is   blue   <eos>  <pad>
─────────────────────────────────────────────────────────────────

Each batch has UNIQUE starting token → Model learns different patterns!

Model processes ALL 3 batches in parallel:
```

```
──────────────────────────────────────────────────────────────────────
                 Batch1       Batch2       Batch3
Input:          [3,4,5,6]    [8,9,10,0]   [11,12,13,0]
                    ↓            ↓            ↓
Embedding:      (4,16)       (4,16)       (4,16)
                    ↓            ↓            ↓
Hidden1+ReLU:   (4,32)       (4,32)       (4,32)
                    ↓            ↓            ↓
Hidden2+ReLU:   (4,32)       (4,32)       (4,32)
                    ↓            ↓            ↓
Logits:         (4,14)       (4,14)       (4,14)
                    ↓            ↓            ↓
Loss:           CE_1         CE_2         CE_3
                    ↓            ↓            ↓
                ──────── Average Loss ────────
──────────────────────────────────────────────────────────────────────

Inference (Autoregressive) - generates ONE token at a time:
──────────────────────────────────────────────────────────────────────
Step 1: [the]              → model → "cat"
Step 2: [the, cat]         → model → "sat"
Step 3: [the, cat, sat]    → model → "on"
Step 4: [the, cat, sat, on] → model → "mat"
──────────────────────────────────────────────────────────────────────
"""
)

# ============================================================
# STEP 9: Summary
# ============================================================
print("=" * 65)
print("Summary")
print("=" * 65)
print(
    """
Transformer-like Model Flow:
┌──────────────────────────────────────────────────────────────────┐
│ Training:                                                        │
│   1. Input tokens → indices → embedding                          │
│   2. Embedding → hidden layers → logits                          │
│   3. Logits vs targets → CrossEntropyLoss                        │
│   4. Backprop → update weights                                   │
│                                                                  │
│ Inference (Autoregressive):                                      │
│   1. Start with <sos>                                            │
│   2. Model predicts next token probabilities                     │
│   3. Sample from probabilities (with temperature)                │
│   4. Append token, repeat until <eos>                            │
│                                                                  │
│ Key Shapes:                                                      │
│   Input:  (batch, sequence)           e.g., (3, 4)               │
│   Embed:  (batch, sequence, embed)    e.g., (3, 4, 16)           │
```

```
│   Logits: (batch, sequence, vocab)    e.g., (3, 4, 14)      │
│   Target: (batch, sequence)           e.g., (3, 4)          │
└─────────────────────────────────────────────────────────────┘

Note: Real transformers use self-attention instead of simple
      linear layers, but the overall flow is the same!
"""
)
```

실행 결과

```
====================================================================
BASIC TRANSFORMER FLOW: Training & Inference
====================================================================

====================================================================
STEP 1: Vocabulary
====================================================================

Vocabulary (14 words):
  0: '<pad>'
  1: '<sos>'
  2: '<eos>'
  3: 'the'
  4: 'cat'
  5: 'sat'
  6: 'on'
  7: 'mat'
  8: 'i'
  9: 'love'
  10: 'pizza'
  11: 'sky'
  12: 'is'
  13: 'blue'

====================================================================
STEP 2: Training Data (batch=3)
====================================================================

Input → Target (next token prediction):

  Batch 1:
    Input:  ['the', 'cat', 'sat', 'on']
    Target: ['cat', 'sat', 'on', 'mat']

  Batch 2:
    Input:  ['i', 'love', 'pizza', '<pad>']
    Target: ['love', 'pizza', '<eos>', '<pad>']

  Batch 3:
    Input:  ['sky', 'is', 'blue', '<pad>']
    Target: ['is', 'blue', '<eos>', '<pad>']
```

```
Tensor shapes:
  X_train: torch.Size([3, 4])  (batch=3, sequence=5)
  Y_train: torch.Size([3, 4])  (batch=3, sequence=5)

X_train (token indices):
tensor([[ 3,  4,  5,  6],
        [ 8,  9, 10,  0],
        [11, 12, 13,  0]])

Y_train (target indices):
tensor([[ 4,  5,  6,  7],
        [ 9, 10,  2,  0],
        [12, 13,  2,  0]])

====================================================================
STEP 3: Simplified Model (2 hidden layers)
====================================================================

Model architecture:
SimplifiedTransformer(
  (embedding): Embedding(14, 16)
  (hidden1): Linear(in_features=16, out_features=32, bias=True)
  (hidden2): Linear(in_features=32, out_features=32, bias=True)
  (output): Linear(in_features=32, out_features=14, bias=True)
  (relu): ReLU()
)

Total parameters: 2286

====================================================================
STEP 4: Training
====================================================================

Loss function: CrossEntropyLoss
Optimizer: Adam (lr=0.01)

Training for 300 epochs...
--------------------------------------------------------------------

  Epoch 50/300
  ├── Loss:     0.0000
  ├── Accuracy: 100.0%
  │   Batch 1: Input: the cat sat on
  │            Output: cat sat on mat ✓
  │   Batch 2: Input: i love pizza
  │            Output: love pizza <eos> ✓
  │   Batch 3: Input: sky is blue
  │            Output: is blue <eos> ✓

  Epoch 100/300
  ├── Loss:     0.0000
  ├── Accuracy: 100.0%
```

```
  │   Batch 1: Input: the cat sat on
  │            Output: cat sat on mat ✓
  │   Batch 2: Input: i love pizza
  │            Output: love pizza <eos> ✓
  │   Batch 3: Input: sky is blue
  │            Output: is blue <eos> ✓

  Epoch 150/300
  ├── Loss:     0.0000
  ├── Accuracy: 100.0%
  │   Batch 1: Input: the cat sat on
  │            Output: cat sat on mat ✓
  │   Batch 2: Input: i love pizza
  │            Output: love pizza <eos> ✓
  │   Batch 3: Input: sky is blue
  │            Output: is blue <eos> ✓

  Epoch 200/300
  ├── Loss:     0.0000
  ├── Accuracy: 100.0%
  │   Batch 1: Input: the cat sat on
  │            Output: cat sat on mat ✓
  │   Batch 2: Input: i love pizza
  │            Output: love pizza <eos> ✓
  │   Batch 3: Input: sky is blue
  │            Output: is blue <eos> ✓

  Epoch 250/300
  ├── Loss:     0.0000
  ├── Accuracy: 100.0%
  │   Batch 1: Input: the cat sat on
  │            Output: cat sat on mat ✓
  │   Batch 2: Input: i love pizza
  │            Output: love pizza <eos> ✓
  │   Batch 3: Input: sky is blue
  │            Output: is blue <eos> ✓

  Epoch 300/300
  ├── Loss:     0.0000
  ├── Accuracy: 100.0%
  │   Batch 1: Input: the cat sat on
  │            Output: cat sat on mat ✓
  │   Batch 2: Input: i love pizza
  │            Output: love pizza <eos> ✓
  │   Batch 3: Input: sky is blue
  │            Output: is blue <eos> ✓

================================================================
STEP 5: Training Results
================================================================
```

```
Predictions vs Targets:
----------------------------------------------------------------

  Batch 1:
    Input:      ['the', 'cat', 'sat', 'on']
    Target:     ['cat', 'sat', 'on', 'mat']
    Predicted:  ['cat', 'sat', 'on', 'mat']
    Match: ✓

  Batch 2:
    Input:      ['i', 'love', 'pizza', '<pad>']
    Target:     ['love', 'pizza', '<eos>', '<pad>']
    Predicted:  ['love', 'pizza', '<eos>', '<pad>']
    Match: ✓

  Batch 3:
    Input:      ['sky', 'is', 'blue', '<pad>']
    Target:     ['is', 'blue', '<eos>', '<pad>']
    Predicted:  ['is', 'blue', '<eos>', '<pad>']
    Match: ✓

================================================================
STEP 6: Inference (Autoregressive Generation)
================================================================

Autoregressive generation:
  1. Start with <sos>
  2. Predict next token
  3. Append predicted token to input
  4. Repeat until <eos> or max length

--- Generation 1 ---
  Starting: ['the']
  Step 1: top3=[('cat', '1.00'), ('pizza', '0.00'), ('<eos>', '0.00')] → 'cat'
  Step 2: top3=[('sat', '1.00'), ('pizza', '0.00'), ('cat', '0.00')] → 'sat'
  Step 3: top3=[('on', '1.00'), ('<eos>', '0.00'), ('<pad>', '0.00')] → 'on'
  Step 4: top3=[('mat', '1.00'), ('<eos>', '0.00'), ('cat', '0.00')] → 'mat'
  Step 5: top3=[('mat', '0.99'), ('<eos>', '0.01'), ('cat', '0.00')] → 'mat'

  Input:  the
  Output: the cat sat on mat mat

--- Generation 2 ---
  Starting: ['i']
  Step 1: top3=[('love', '1.00'), ('on', '0.00'), ('sat', '0.00')] → 'love'
  Step 2: top3=[('pizza', '1.00'), ('cat', '0.00'), ('sat', '0.00')] → 'pizza'
  Step 3: top3=[('<eos>', '1.00'), ('mat', '0.00'), ('cat', '0.00')] → '<eos>'

  Input:  i
  Output: i love pizza

--- Generation 3 ---
  Starting: ['sky']
  Step 1: top3=[('is', '1.00'), ('<pad>', '0.00'), ('<eos>', '0.00')] → 'is'
```

```
  Step 2: top3=[('blue', '1.00'), ('<pad>', '0.00'), ('love', '0.00')] → 'blue'
  Step 3: top3=[('<eos>', '1.00'), ('on', '0.00'), ('cat', '0.00')] → '<eos>'

  Input:  sky
  Output: sky is blue

================================================================
Inference Summary:
================================================================
  Input: the   →   Output: the cat sat on mat mat
  Input: i     →   Output: i love pizza
  Input: sky   →   Output: sky is blue

================================================================
STEP 7: Data Flow Visualization
================================================================

Training Data Flow (3 different sentences in one batch):
────────────────────────────────────────────────────────────────
Batch 1: "the cat sat on mat"
  Input:   the    cat    sat    on
  Target:  cat    sat    on    mat

Batch 2: "i love pizza"
  Input:    i    love   pizza  <pad>
  Target:  love  pizza  <eos>  <pad>

Batch 3: "sky is blue"
  Input:   sky    is    blue   <pad>
  Target:   is   blue   <eos>  <pad>
────────────────────────────────────────────────────────────────

Each batch has UNIQUE starting token → Model learns different patterns!

Model processes ALL 3 batches in parallel:
────────────────────────────────────────────────────────────────
                Batch1       Batch2       Batch3
Input:         [3,4,5,6]    [8,9,10,0]   [11,12,13,0]
                   ↓            ↓            ↓
Embedding:     (4,16)       (4,16)       (4,16)
                   ↓            ↓            ↓
Hidden1+ReLU:  (4,32)       (4,32)       (4,32)
                   ↓            ↓            ↓
Hidden2+ReLU:  (4,32)       (4,32)       (4,32)
                   ↓            ↓            ↓
Logits:        (4,14)       (4,14)       (4,14)
                   ↓            ↓            ↓
Loss:          CE_1         CE_2         CE_3
                   ↓            ↓            ↓
               ──────── Average Loss ────────
────────────────────────────────────────────────────────────────

Inference (Autoregressive) - generates ONE token at a time:
────────────────────────────────────────────────────────────────
```

```
Step 1: [the]              → model → "cat"
Step 2: [the, cat]         → model → "sat"
Step 3: [the, cat, sat]    → model → "on"
Step 4: [the, cat, sat, on] → model → "mat"
────────────────────────────────────────────────────────────────

================================================================
Summary
================================================================

Transformer-like Model Flow:

┌──────────────────────────────────────────────────────────┐
│ Training:                                                │
│   1. Input tokens → indices → embedding                  │
│   2. Embedding → hidden layers → logits                  │
│   3. Logits vs targets → CrossEntropyLoss                │
│   4. Backprop → update weights                           │
│                                                          │
│ Inference (Autoregressive):                              │
│   1. Start with <sos>                                    │
│   2. Model predicts next token probabilities             │
│   3. Sample from probabilities (with temperature)        │
│   4. Append token, repeat until <eos>                    │
│                                                          │
│ Key Shapes:                                              │
│   Input:  (batch, sequence)           e.g., (3, 4)       │
│   Embed:  (batch, sequence, embed)    e.g., (3, 4, 16)   │
│   Logits: (batch, sequence, vocab)    e.g., (3, 4, 14)   │
│   Target: (batch, sequence)           e.g., (3, 4)       │
└──────────────────────────────────────────────────────────┘

Note: Real transformers use self-attention instead of simple
      linear layers, but the overall flow is the same!
```

부록: 빠른 참조

분류 vs 회귀

작업	출력 활성화	손실 함수	옵티마이저
이진 분류	Sigmoid	BCELoss	SGD/Adam
다중 클래스 분류	Softmax	CrossEntropyLoss	Adam
회귀	없음 (선형)	MSELoss	Adam

학습 루프 패턴

```python
for epoch in range(epochs):
    predictions = model(X)            # 순전파
    loss = criterion(predictions, y) # 손실 계산
    optimizer.zero_grad()            # 그래디언트 초기화
```

```
    loss.backward()                     # 역전파
    optimizer.step()                    # 가중치 업데이트
```

추론 패턴

```
model.eval()                            # 평가 모드 설정
with torch.no_grad():                   # 그래디언트 비활성화
    output = model(input)               # 순전파
    prediction = output.argmax()        # 클래스 얻기 (분류)
```